arXiv:2301.04104v2 [cs.AI] 17 Apr 2024

Mastering Diverse Domains through World Models

Danijar Hafner[1,2] Jurgis Pasukonis[1] Jimmy Ba[2] Timothy Lillicrap[1]

Abstract

Developing a general algorithm that learns to solve tasks across a wide range of applications has been a fundamental challenge in artificial intelligence. Although current reinforcement learning algorithms can be readily applied to tasks similar to what they have been developed for, configuring them for new application domains requires significant human expertise and experimentation. We present DreamerV3, a general algorithm that outperforms specialized methods across over 150 diverse tasks, with a single configuration. Dreamer learns a model of the environment and improves its behavior by imagining future scenarios. Robustness techniques based on normalization, balancing, and transformations enable stable learning across domains. Applied out of the box, Dreamer is the first algorithm to collect diamonds in Minecraft from scratch without human data or curricula. This achievement has been posed as a significant challenge in artificial intelligence that requires exploring farsighted strategies from pixels and sparse rewards in an open world. Our work allows solving challenging control problems without extensive experimentation, making reinforcement learning broadly applicable.

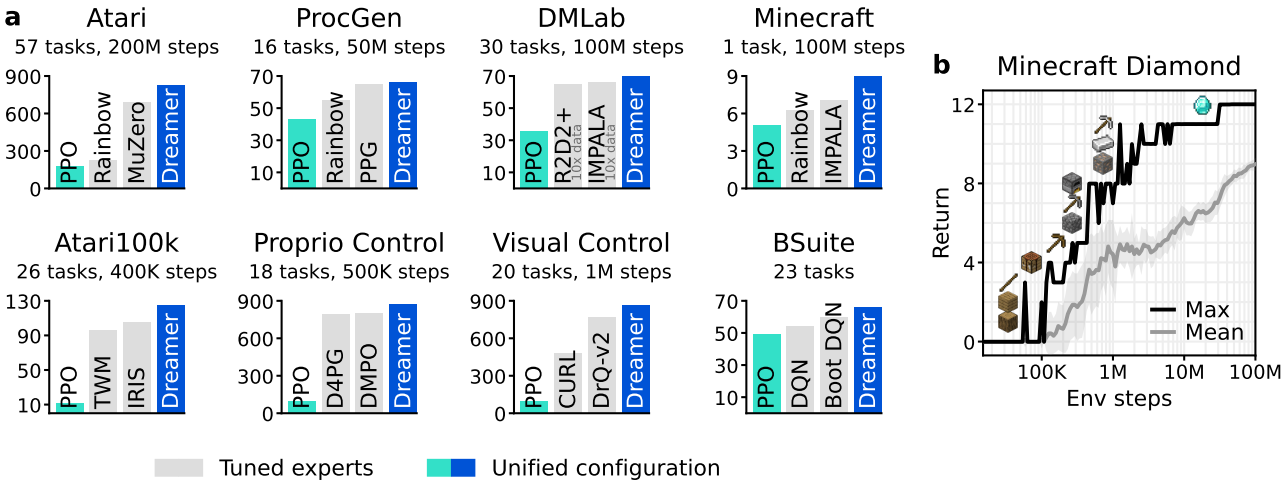
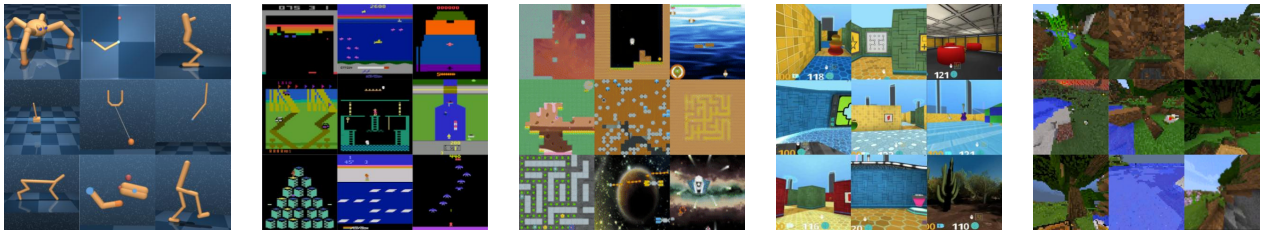


Figure 1: Benchmark summary. **a**, Using fixed hyperparameters across all domains, Dreamer outperforms tuned expert algorithms across a wide range of benchmarks and data budgets. Dreamer also substantially outperforms a high-quality implementation of the widely applicable PPO algorithm. **b**, Applied out of the box, Dreamer learns to obtain diamonds in the popular video game Minecraft from scratch given sparse rewards, a long-standing challenge in artificial intelligence for which previous approaches required human data or domain-specific heuristics.

[1]Google DeepMind. [2]University of Toronto. Correspondence: mail@danijar.com



(a) Control Suite **(b)** Atari **(c)** ProcGen **(d)** DMLab **(e)** Minecraft

Figure 2: Diverse visual domains used in the experiments. Dreamer succeeds across these domains, ranging from robot locomotion and manipulation tasks over Atari games, procedurally generated ProcGen levels, and DMLab tasks, that require spatial and temporal reasoning, to the complex and infinite world of Minecraft. We also evaluate Dreamer on non-visual domains.

Introduction

Reinforcement learning has enabled computers to solve tasks through interaction, such as surpassing humans in the games of Go and Dota[1,2]. It is also a key component for improving large language models beyond what is demonstrated in their pretraining data[3,4]. While PPO[5] has become a standard algorithm in the field of reinforcement learning, more specialized algorithms are often employed to achieve higher performance. These specialized algorithms target the unique challenges posed by different application domains, such as continuous control[6], discrete actions[7,8], sparse rewards[9], image inputs[10], spatial environments[11], and board games[12]. However, applying reinforcement learning algorithms to sufficiently new tasks—such as moving from video games to robotics tasks—requires substantial effort, expertise, and computational resources for tweaking the hyperparameters of the algorithm[13]. This brittleness poses a bottleneck in applying reinforcement learning to new problems and also limits the applicability of reinforcement learning to computationally expensive models or tasks where tuning is prohibitive. Creating a general algorithm that learns to master new domains without having to be reconfigured has been a central challenge in artificial intelligence and would open up reinforcement learning to a wide range of practical applications.

We present Dreamer, a general algorithm that outperforms specialized expert algorithms across a wide range of domains while using fixed hyperparameters, making reinforcement learning readily applicable to new problems. The algorithm is based on the idea of learning a world model that equips the agent with rich perception and the ability to imagine the future[14,15,16]. The world model predicts the outcomes of potential actions, a critic neural network judges the value of each outcome, and an actor neural network chooses actions to reach the best outcomes. Although intuitively appealing, robustly learning and leveraging world models to achieve strong task performance has been an open problem[17]. Dreamer overcomes this challenge through a range of robustness techniques based on normalization, balancing, and transformations. We observe robust learning not only across over 150 tasks from the domains summarized in Figure 2, but also across model sizes and training budgets, offering a predictable way to increase performance. Notably, larger model sizes not only achieve higher scores but also require less interaction to solve a task.

To push the boundaries of reinforcement learning, we consider the popular video game Minecraft that has become a focal point of research in recent years[18,19,20], with international competitions held for developing algorithms that autonomously learn to collect diamonds in Minecraft*. Solving this

*The MineRL Diamond Competitions were held in 2019, 2020, and 2021 and provided a dataset of human expert trajectories: `https://minerl.io/diamond`. Competitions in the following years focused on a wide range of tasks.

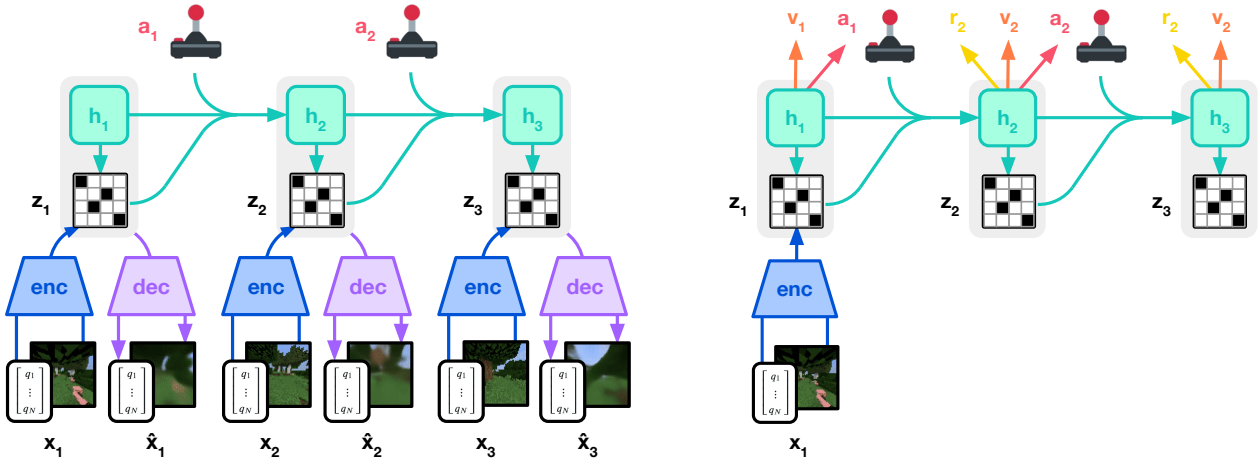


Figure 3: Training process of Dreamer. The world model encodes sensory inputs into discrete representations z_t that are predicted by a sequence model with recurrent state h_t given actions a_t. The inputs are reconstructed to shape the representations. The actor and critic predict actions a_t and values v_t and learn from trajectories of abstract representations predicted by the world model.

problem without human data has been widely recognized as a substantial challenge for artificial intelligence because of the sparse rewards, exploration difficulty, long time horizons, and the procedural diversity of this open world game[18]. Due to these obstacles, previous approaches resorted to using human expert data and domain-specific curricula[19,20]. Applied out of the box, Dreamer is the first algorithm to collect diamonds in Minecraft from scratch.

Learning algorithm

We present the third generation of the Dreamer algorithm[21,22]. The algorithm consists of three neural networks: the world model predicts the outcomes of potential actions, the critic judges the value of each outcome, and the actor chooses actions to reach the most valuable outcomes. The components are trained concurrently from replayed experience while the agent interacts with the environment. To succeed across domains, all three components need to accommodate different signal magnitudes and robustly balance terms in their objectives. This is challenging as we are not only targeting similar tasks within the same domain but aim to learn across diverse domains with fixed hyperparameters. This section introduces the world model, critic, and actor along with their robust loss functions, as well as tools for robustly predicting quantities of unknown orders of magnitude.

World model learning

The world model learns compact representations of sensory inputs through autoencoding[23] and enables planning by predicting future representations and rewards for potential actions. We implement the world model as a Recurrent State-Space Model (RSSM)[24], shown in Figure 3. First, an encoder maps sensory inputs x_t to stochastic representations z_t. Then, a sequence model with recurrent state h_t predicts the sequence of these representations given past actions a_{t-1}. The concatenation of

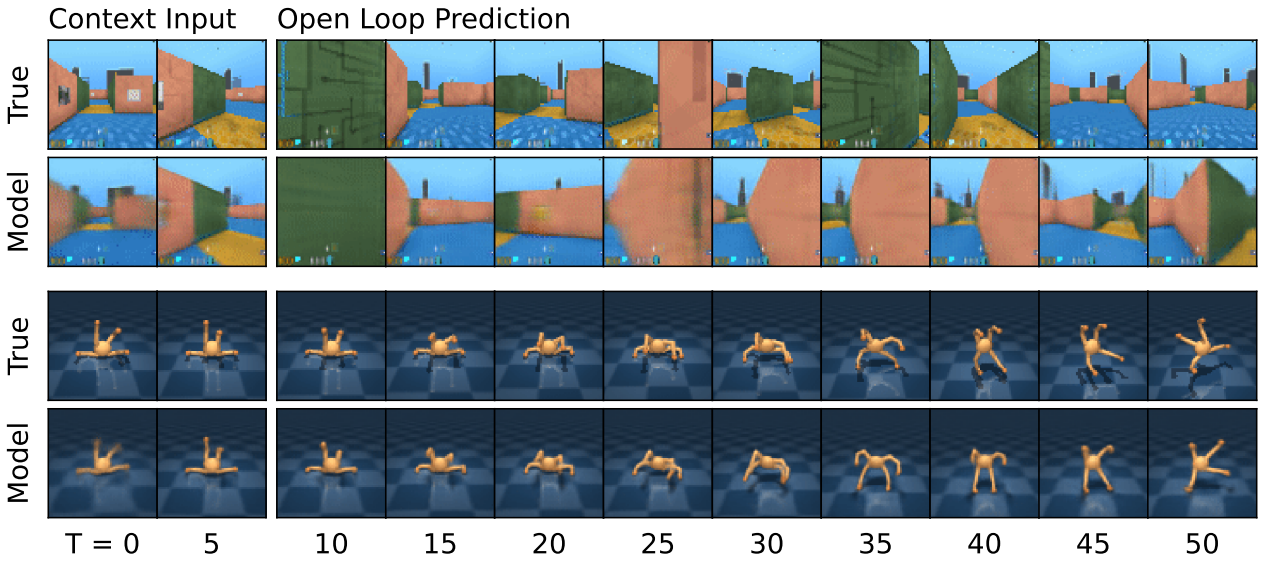


Figure 4: Multi-step video predictions of a DMLab maze (top) and a quadrupedal robot (bottom). Given 5 context images and the full action sequence, the model predicts 45 frames into the future without access to intermediate images. The world model learns an understanding of the underlying structure of each environment.

h_t and z_t forms the model state from which we predict rewards r_t and episode continuation flags $c_t \in \{0, 1\}$ and reconstruct the inputs to ensure informative representations:

$$\text{RSSM} \begin{cases} \text{Sequence model:} & h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1}) \\ \text{Encoder:} & z_t \sim q_\phi(z_t \mid h_t, x_t) \\ \text{Dynamics predictor:} & \hat{z}_t \sim p_\phi(\hat{z}_t \mid h_t) \\ \text{Reward predictor:} & \hat{r}_t \sim p_\phi(\hat{r}_t \mid h_t, z_t) \\ \text{Continue predictor:} & \hat{c}_t \sim p_\phi(\hat{c}_t \mid h_t, z_t) \\ \text{Decoder:} & \hat{x}_t \sim p_\phi(\hat{x}_t \mid h_t, z_t) \end{cases} \tag{1}$$

Figure 4 visualizes long-term video predictions of the world world. The encoder and decoder use convolutional neural networks (CNN) for image inputs and multi-layer perceptrons (MLPs) for vector inputs. The dynamics, reward, and continue predictors are also MLPs. The representations are sampled from a vector of softmax distributions and we take straight-through gradients through the sampling step[25,22]. Given a sequence batch of inputs $x_{1:T}$, actions $a_{1:T}$, rewards $r_{1:T}$, and continuation flags $c_{1:T}$, the world model parameters ϕ are optimized end-to-end to minimize the prediction loss $\mathcal{L}_{\text{pred}}$, the dynamics loss $\mathcal{L}_{\text{dyn}}$, and the representation loss $\mathcal{L}_{\text{rep}}$ with corresponding loss weights $\beta_{\text{pred}} = 1$, $\beta_{\text{dyn}} = 1$, and $\beta_{\text{rep}} = 0.1$:

$$\mathcal{L}(\phi) \doteq \mathrm{E}_{q_\phi}\Big[\textstyle\sum_{t=1}^{T}(\beta_{\text{pred}}\mathcal{L}_{\text{pred}}(\phi) + \beta_{\text{dyn}}\mathcal{L}_{\text{dyn}}(\phi) + \beta_{\text{rep}}\mathcal{L}_{\text{rep}}(\phi))\Big]. \tag{2}$$

The prediction loss trains the decoder and reward predictor via the symlog squared loss described later, and the continue predictor via logistic regression. The dynamics loss trains the sequence model to predict the next representation by minimizing the KL divergence between the predictor $p_\phi(z_t \mid h_t)$ and the next stochastic representation $q_\phi(z_t \mid h_t, x_t)$. The representation loss, in turn, trains the representations to become more predictable allowing us to use a factorized dynamics predictor for fast sampling during imagination training. The two losses differ in the stop-gradient operator $\mathrm{sg}(\cdot)$ and their loss scale. To avoid a degenerate solution where the dynamics are trivial to predict but fail

to contain enough information about the inputs, we employ free bits[26] by clipping the dynamics and representation losses below the value of 1 nat ≈ 1.44 bits. This disables them while they are already minimized well to focus learning on the prediction loss:

$$\begin{aligned}
\mathcal{L}_{\text{pred}}(\phi) &\doteq -\ln p_\phi(x_t \mid z_t, h_t) - \ln p_\phi(r_t \mid z_t, h_t) - \ln p_\phi(c_t \mid z_t, h_t) \\
\mathcal{L}_{\text{dyn}}(\phi) &\doteq \max\big(1, \mathrm{KL}\big[\,\mathrm{sg}(q_\phi(z_t \mid h_t, x_t)) \;\big\|\; p_\phi(z_t \mid h_t)\,\big]\big) \\
\mathcal{L}_{\text{rep}}(\phi) &\doteq \max\big(1, \mathrm{KL}\big[\,q_\phi(z_t \mid h_t, x_t) \;\big\|\; \mathrm{sg}(p_\phi(z_t \mid h_t))\big]\big)
\end{aligned} \tag{3}$$

Previous world models require scaling the representation loss differently based on the visual complexity of the environment[21]. Complex 3D environments contain details unnecessary for control and thus prompt a stronger regularizer to simplify the representations and make them more predictable. In games with static backgrounds and where individual pixels may matter for the task, a weak regularizer is required to extract fine details. We find that combining free bits with a small representation loss resolves this dilemma, allowing for fixed hyperparameters across domains. Moreover, we transform vector observations using the symlog function described later, to prevent large inputs and large reconstruction gradients, further stabilizing the trade-off with the representation loss.

We occasionally observed spikes the in KL losses in earlier experiments, consistent with reports for deep variational autoencoders[27]. To prevent this, we parameterize the categorical distributions of the encoder and dynamics predictor as mixtures of 1% uniform and 99% neural network output, making it impossible for them to become deterministic and thus ensuring well-behaved KL losses. Further model details and hyperparameters are included in the supplementary material.

Critic learning

The actor and critic neural networks learn behaviors purely from abstract trajectories of representations predicted by the world model[14]. For environment interaction, we select actions by sampling from the actor network without lookahead planning. The actor and critic operate on model states $s_t \doteq \{h_t, z_t\}$ and thus benefit from the Markovian representations learned by the recurrent world model. The actor aims to maximize the return $R_t \doteq \sum_{\tau=0}^{\infty} \gamma^\tau r_{t+\tau}$ with a discount factor $\gamma = 0.997$ for each model state. To consider rewards beyond the prediction horizon $T = 16$, the critic learns to approximate the distribution of returns[28] for each state under the current actor behavior:

$$\text{Actor:} \qquad a_t \sim \pi_\theta(a_t \mid s_t) \qquad\qquad \text{Critic:} \qquad v_\psi(R_t \mid s_t) \tag{4}$$

Starting from representations of replayed inputs, the world model and actor generate a trajectory of imagined model states $s_{1:T}$, actions $a_{1:T}$, rewards $r_{1:T}$, and continuation flags $c_{1:T}$. Because the critic predicts a distribution, we read out its predicted values $v_t \doteq \mathrm{E}[v_\psi(\,\cdot \mid s_t)]$ as the expectation of the distribution. To estimate returns that consider rewards beyond the prediction horizon, we compute bootstrapped λ-returns[29] that integrate the predicted rewards and the values. The critic learns to predict the distribution of the return estimates R_t^λ using the maximum likelihood loss:

$$\mathcal{L}(\psi) \doteq -\sum_{t=1}^{T} \ln p_\psi(R_t^\lambda \mid s_t) \qquad R_t^\lambda \doteq r_t + \gamma c_t \Big((1-\lambda) v_t + \lambda R_{t+1}^\lambda\Big) \qquad R_T^\lambda \doteq v_T \tag{5}$$

While a simple choice would be to parameterize the critic as a Normal distribution, the return distribution can have multiple modes and vary by orders of magnitude across environments. To stabilize and accelerate learning under these conditions, we parameterize the critic as categorical distribution with exponentially spaced bins, decoupling the scale of gradients from the prediction

targets as described later. To improve value prediction in environments where rewards are challenging to predict, we apply the critic loss both to imagined trajectories with loss scale $\beta_{\text{val}} = 1$ and to trajectories sampled from the replay buffer with loss scale $\beta_{\text{repval}} = 0.3$. The critic replay loss uses the imagination returns R_t^λ at the start states of the imagination rollouts as on-policy value annotations for the replay trajectory to then compute λ-returns over the replay rewards.

Because the critic regresses targets that depend on its own predictions, we stabilize learning by regularizing the critic towards predicting the outputs of an exponentially moving average of its own parameters. This is similar to target networks used previously in reinforcement learning[7] but allows us to compute returns using the current critic network. We further noticed that the randomly initialized reward predictor and critic networks at the start of training can result in large predicted rewards that can delay the onset of learning. We thus initialize the output weight matrix of the reward predictor and critic to zeros, which alleviates the problem and accelerates early learning.

Actor learning

The actor learns to choose actions that maximize return while exploring through an entropy regularizer[30]. However, the correct scale for this regularizer depends both on the scale and frequency of rewards in the environment. Ideally, we would like the agent to explore more if rewards are sparse and exploit more if rewards are dense or nearby. At the same time, the exploration amount should not be influenced by arbitrary scaling of rewards in the environment. This requires normalizing the return scale while preserving information about reward frequency.

To use a fixed entropy scale of $\eta = 3 \times 10^{-4}$ across domains, we normalize returns to be approximately contained in the interval $[0, 1]$. In practice, substracting an offset from the returns does not change the actor gradient and thus dividing by the range S is sufficient. Moreover, to avoid amplifying noise from function approximation under sparse rewards, we only scale down large return magnitudes but leave small returns below the threshold of $L = 1$ untouched. We use the Reinforce estimator[31] for both discrete and continuous actions, resulting in the surrogate loss function:

$$\mathcal{L}(\theta) \doteq -\sum_{t=1}^{T} \operatorname{sg}\Big(\big(R_t^\lambda - v_\psi(s_t)\big) / \max(1,\, S)\Big) \log \pi_\theta(a_t \mid s_t) + \eta \operatorname{H}\big[\pi_\theta(a_t \,\big|\, s_t)\big] \tag{6}$$

The return distribution can be multi-modal and include outliers, especially for randomized environments where some episodes have higher achievable returns than others. Normalizing by the smallest and largest observed returns would then scale returns down too much and may cause suboptimal convergence. To be robust to these outliers, we compute the range from the 5th to the 95th return percentile over the return batch and smooth out the estimate using an exponential moving average:

$$S \doteq \operatorname{EMA}\big(\operatorname{Per}(R_t^\lambda, 95) - \operatorname{Per}(R_t^\lambda, 5), 0.99\big) \tag{7}$$

Previous work typically normalizes advantages[5] rather than returns, which puts a fixed amount of emphasis on maximizing returns over entropy regardless of whether rewards are within reach. Scaling up advantages when rewards are sparse can amplify noise that outweighs the entropy regularizer and stagnates exploration. Normalizing rewards or returns by standard deviation can fail under sparse rewards where their standard deviation is near zero, drastically amplifying rewards regardless of their size. Constrained optimization targets a fixed entropy on average across states[32,33] regardless of achievable returns, which is robust but explores slowly under sparse rewards and converges lower under dense rewards. We did not find stable hyperparameters across domains for these approaches. Return normalization with a denominator limit overcomes these challenges, exploring rapidly under sparse rewards and converging to high performance across diverse domains.

Robust predictions

Reconstructing inputs and predicting rewards and returns can be challenging because the scale of these quantities can vary across domains. Predicting large targets using a squared loss can lead to divergence whereas absolute and Huber losses[7] stagnate learning. On the other hand, normalizing targets based on running statistics[5] introduces non-stationarity into the optimization. We suggest the symlog squared error as a simple solution to this dilemma. For this, a neural network $f(x, \theta)$ with inputs x and parameters θ learns to predict a transformed version of its targets y. To read out predictions $\hat{y}$ of the network, we apply the inverse transformation:

$$\mathcal{L}(\theta) \doteq \tfrac{1}{2}\big(f(x,\theta) - \operatorname{symlog}(y)\big)^2 \qquad \hat{y} \doteq \operatorname{symexp}\big(f(x,\theta)\big) \tag{8}$$

Using the logarithm as transformation would not allow us to predict targets that take on negative values. Therefore, we choose a function from the bi-symmetric logarithmic family[34] that we name symlog as the transformation with the symexp function as its inverse:

$$\operatorname{symlog}(x) \doteq \operatorname{sign}(x) \ln\big(|x| + 1\big) \qquad \operatorname{symexp}(x) \doteq \operatorname{sign}(x)\big(\exp(|x|) - 1\big) \tag{9}$$

The symlog function compresses the magnitudes of both large positive and negative values. Unlike the logarithm, it is symmetric around the origin while preserving the input sign. This allows the optimization process to quickly move the network predictions to large values when needed. The symlog function approximates the identity around the origin so that it does not affect learning of targets that are already small enough.

For potentially stochastic targets, such as rewards or returns, we introduce the symexp twohot loss. Here, the network outputs the logits for a softmax distribution over exponentially spaced bins $b_i \in B$. Predictions are read out as the weighted average of the bin positions weighted by their predicted probabilities. Importantly, the network can output any continuous value in the interval because the weighted average can fall between the buckets:

$$\hat{y} \doteq \operatorname{softmax}(f(x))^T B \qquad B \doteq \operatorname{symexp}(\begin{bmatrix}-20 & \dots & +20\end{bmatrix}) \tag{10}$$

The network is trained on twohot encoded targets[8,28], a generalization of onehot encoding to continuous values. The twohot encoding of a scalar is a vector with $|B|$ entries that are all 0 except at the indices k and $k+1$ of the two bins closest to the encoded scalar. The two entries sum up to 1, with linearly higher weight given to the bin that is closer to the encoded continuous number. The network is then trained to minimize the categorical cross entropy loss for classification with soft targets. Note that the loss only depends on the probabilities assigned to the bins but not on the continuous values associated with the bin locations, decoupling the size of the gradients from the size of the targets:

$$\mathcal{L}(\theta) \doteq -\operatorname{twohot}(y)^T \log \operatorname{softmax}(f(x,\theta)) \tag{11}$$

Applying these principles, Dreamer transforms vector observations using the symlog functions, both for the encoder inputs and the decoder targets and employs the synexp twohot loss for the reward predictor and critic. We find that these techniques enable robust and fast learning across many diverse domains. For critic learning, an alternative asymmetric transformation has previously been proposed[35], which we found less effective on average across domains. Unlike alternatives, symlog transformations avoid truncating large targets[7], introducing non-stationary from normalization[5], or adjusting network weights when new extreme values are detected[36].

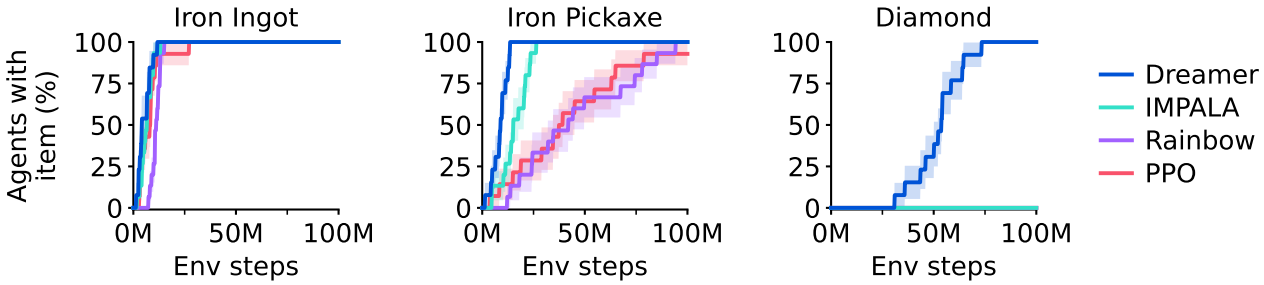


Figure 5: Fraction of trained agents that discover each of the three latest items in the Minecraft Diamond task. Although previous algorithms progress up to the iron pickaxe, Dreamer is the only compared algorithm that manages to discover a diamond, and does so reliably.

Results

We evaluate the generality of Dreamer across 8 domains—with over 150 tasks—under fixed hyperparameters. We designed the experiments to compare Dreamer to the best methods in the literature, which are often specifically designed and tuned for the benchmark at hand. We further compare to a high-quality implementation of PPO[5], a standard reinforcement learning algorithm that is known for its robustness. We run PPO with fixed hyperparameters chosen to maximize performance across domains and that reproduce strong published results of PPO on ProcGen[37]. To push the boundaries of reinforcement learning, we apply Dreamer to the challenging video game Minecraft, comparing it to strong previous algorithms. Finally, we analyze the importance of individual components of Dreamer and its robustness to different model sizes and computational budgets. All Dreamer agents are trained on a single Nvidia A100 GPU each, making it reproducible for many research labs. A public implementation of Dreamer that reproduces all results is available on the project website.

Benchmarks We perform an extensive empirical study across 8 domains that include continuous and discrete actions, visual and low-dimensional inputs, dense and sparse rewards, different reward scales, 2D and 3D worlds, and procedural generation. Figure 1 summarizes the benchmark results, showing that Dreamer outperforms a wide range of previous expert algorithms across diverse domains. Crucially, Dreamer substantially outperforms PPO across all domains.

- **Atari** This established benchmark contains 57 Atari 2600 games with a budget of 200M frames, posing a diverse range of challenges[38]. We use the sticky action simulator setting[39]. Dreamer outperforms the powerful MuZero algorithm[8] while using only a fraction of the computational resources. Dreamer also outperforms the widely-used expert algorithms Rainbow[40] and IQN[41].
- **ProcGen** This benchmark of 16 games features randomized levels and visual distractions to test the robustness and generalization of agents[42]. Within the budget of 50M frames, Dreamer matches the tuned expert algorithm PPG[37] and outperforms Rainbow[42,40]. Our PPO agent with fixed hyperparameters matches the published score of the highly tuned official PPO implementation[37].
- **DMLab** This suite of 30 tasks features 3D environments that test spatial and temporal reasoning[43]. In 100M frames, Dreamer exceeds the performance of the scalable IMPALA and R2D2+ agents[35] at 1B environment steps, amounting to a data-efficiency gain of over 1000%. We note that these baselines were not designed for data-efficiency but serve as a valuable comparison point for the performance previously achievable at scale.

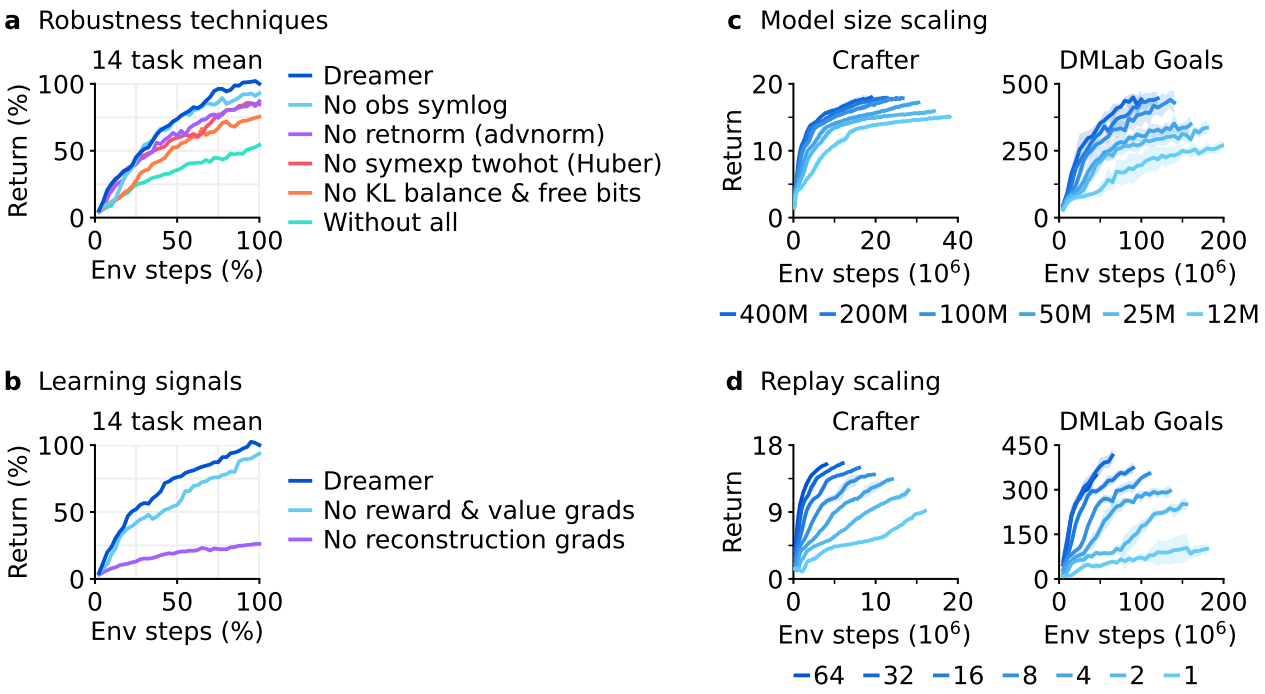


Figure 6: Ablations and robust scaling of Dreamer. **a**, All individual robustness techniques contribute to the performance of Dreamer on average, although each individual technique may only affect some tasks. Training curves of individual tasks are included in the supplementary material. **b**, The performance of Dreamer predominantly rests on the unsupervised reconstruction loss of its world model, unlike most prior algorithms that rely predominantly on reward and value prediction gradients[7,5,8]. **c**, The performance of Dreamer increases monotonically with larger model sizes, ranging from 12M to 400M parameters. Notably, larger models not only increase task performance but also require less environment interaction. **d**, Higher replay ratios predictably increase the performance of Dreamer. Together with model size, this allows practitioners to improve task performance and data-efficiency by employing more computational resources.

- **Atari100k** This data-efficiency benchmark comntains 26 Atari games and a budget of only 400K frames, amounting to 2 hours of game time[17]. EfficientZero[44] holds the state-of-the-art by combining online tree search, prioritized replay, and hyperparameter scheduling, but also resets levels early to increase data diversity, making a comparison difficult. Without this complexity, Dreamer outperforms the best remaining methods, including the transformer-based IRIS and TWM agents, the model-free SPR, and SimPLe[45].
- **Proprio Control** This benchmark contains 18 control tasks with continuous actions, proprioceptive vector inputs, and a budget of 500K environment steps[46]. The tasks range from classical control over locomotion to robot manipulation tasks, featuring dense and sparse rewards. Dreamer sets a new state-of-the-art on this benchmark, outperforming D4PG, DMPO, and MPO[33].
- **Visual Control** This benchmark consists of 20 continuous control tasks where the agent receives only high-dimensional images as input and has a budget of 1M environment steps[46]. Dreamer establishes a new state-of-the-art on this benchmark, outperforming DrQ-v2 and CURL[47], which are specialized to visual environments and leverage data augmentation.

- **BSuite** This benchmark includes 23 environments with a total of 468 configurations that are specifically designed to test credit assignment, robustness to reward scale and stochasticity, memory, generalization, and exploration[48]. Dreamer establishes a new state-of-the-art on this benchmark, outperforming Boot DQN and other methods[49]. Dreamer improves over previous algorithms especially in the scale robustness category.

Minecraft Collecting diamonds in the popular game Minecraft has been a long-standing challenge in artificial intelligence[18,19,20]. Every episode in this game is set in a unique randomly generated and infinite 3D world. Episodes last until the player dies or up to 36000 steps equaling 30 minutes, during which the player needs to discover a sequence of 12 items from sparse rewards by foraging for resources and crafting tools. It takes about 20 minutes for experienced human players to obtain diamonds[20]. We follow the block breaking setting of prior work[19] because the provided action space would make it challenging for stochastic policies to keep a key pressed for a prolonged time.

Because of the training time in this complex domain, extensive tuning would be difficult for Minecraft. Instead, we apply Dreamer out of the box with its default hyperparameters. As shown in Figures 1 and 5, Dreamer is the first algorithm to collect diamonds in Minecraft from scratch without using human data as was required by VPT[20] or adaptive curricula[19]. All the Dreamer agents we trained on Minecraft discover diamonds in 100M environment steps. While several strong baselines progress to advanced items such as the iron pickaxe, none of them discovers a diamond.

Ablations In Figure 6, we ablate the robustness techniques and learning signals on a diverse set of 14 tasks to understand their importance. The training curves of individual tasks are included in the supplementary material. We observe that all robustness techniques contribute to performance, most notably the KL objective of the world model, followed by return normalization and symexp twohot regression for reward and value prediction. In general, we find that each individual technique is critical on a subset of tasks but may not affect performance on other tasks.

To investigate the effect of the world model, we ablate the learning signals of Dreamer by stopping either the task-specific reward and value prediction gradients or the task-agnostic reconstruction gradients from shaping its representations. Unlike previous reinforcement learning algorithms that often rely only on task-specific learning signals[7,8], Dreamer rests predominantly on the unsupervised objective of its world model. This finding could allow for future algorithm variants that leverage pretraining on unsupervised data.

Scaling properties To investigate whether Dreamer can scale robustly, we train 6 model sizes ranging from 12M to 400M parameters, as well as different replay ratios on Crafter[50] and a DMLab task[43]. The replay ratio affects the number of gradient updates performed by the agent. Figure 6 shows robust learning with fixed hyperparameters across the compared model sizes and replay ratios. Moreover, increasing the model size directly translates to both higher task performance and a lower data requirement. Increasing the number of gradient steps further reduces the interactions needed to learn successful behaviors. The results show that Dreamer learns robustly across model sizes and replay ratios and that its performance and provides a predictable way for increasing performance given computational resources.

Previous work

Developing general-purpose algorithms has long been a goal of reinforcement learning research. PPO[5] is one of the most widely used algorithms and is relatively robust but requires large amounts of experience and often yields lower performance than specialized alternatives. SAC[32] is a popular choice for continuous control and leverages experience replay for data-efficiency, but in practice requires tuning, especially for its entropy scale, and struggles under high-dimensional inputs[51]. MuZero[8] plans using a value prediction model and has been applied to board games and Atari, but the authors did not release an implementation and the algorithm contains several complex components, making it challenging to reproduce. Gato[52] fits one large model to expert demonstrations of multiple tasks, but is only applicable when expert data is available. In comparison, Dreamer masters a diverse range of environments with fixed hyperparameters, does not require expert data, and its implementation is open source.

Minecraft has been a focus of recent research. With MALMO[53], Microsoft released a free version of the successful game for research purposes. MineRL[18] offers several competition environments, which we rely on as the basis for our experiments. The MineRL competition supports agents in exploring and learning meaningful skills through a diverse human dataset[18]. Voyager obtains items at a similar depth in the technology tree as Dreamer using API calls to a language model but operates on top of the MineFlayer bot scripting layer that was specifically engineered to the game and exposes high-level actions[54]. VPT[20] trained an agent to play Minecraft through behavioral cloning based on expert data of keyboard and mouse actions collected by contractors and finetuning using reinforcement learning to obtain diamonds using 720 GPUs for 9 days. In comparison, Dreamer uses the MineRL competition action space to autonomously learn to collect diamonds from sparse rewards using 1 GPU for 9 days, without human data.

Conclusion

We present the third generation of the Dreamer algorithm, a general reinforcement learning algorithm that masters a wide range of domains with fixed hyperparameters. Dreamer excels not only across over 150 tasks but also learns robustly across varying data and compute budgets, moving reinforcement learning toward a wide range of practical applications. Applied out of the box, Dreamer is the first algorithm to collect diamonds in Minecraft from scratch, achieving a significant milestone in the field of artificial intelligence. As a high-performing algorithm that is based on a learned world model, Dreamer paves the way for future research directions, including teaching agents world knowledge from internet videos and learning a single world model across domains to allow artificial agents to build up increasingly general knowledge and competency.

Acknowledgements We thank Mohammad Norouzi, Jessy Lin, Abbas Abdolmaleki, John Schulman, Adam Kosiorek, and Oleh Rybkin for insightful discussions. We thank Bobak Shahriari, Denis Yarats, Karl Cobbe, and Hubert Soyer for sharing training curves of baseline algorithms. We thank Daniel Furrer, Andrew Chen, and Dakshesh Garambha for help with Google Cloud infrastructure.

References

1. David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587): 484, 2016.
2. OpenAI. OpenAI Five. https://blog.openai.com/openai-five/, 2018.
3. Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
4. Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu Hong Hoi. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *Advances in Neural Information Processing Systems*, 35:21314–21328, 2022.
5. John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
6. Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
7. Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529, 2015.
8. Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. Mastering atari, go, chess and shogi by planning with a learned model. *arXiv preprint arXiv:1911.08265*, 2019.
9. Max Jaderberg, Volodymyr Mnih, Wojciech Marian Czarnecki, Tom Schaul, Joel Z Leibo, David Silver, and Koray Kavukcuoglu. Reinforcement learning with unsupervised auxiliary tasks. *arXiv preprint arXiv:1611.05397*, 2016.
10. Ankesh Anand, Evan Racah, Sherjil Ozair, Yoshua Bengio, Marc-Alexandre Côté, and R Devon Hjelm. Unsupervised state representation learning in atari. *Advances in neural information processing systems*, 32, 2019.
11. Danny Driess, Ingmar Schubert, Pete Florence, Yunzhu Li, and Marc Toussaint. Reinforcement learning with neural radiance fields. *arXiv preprint arXiv:2206.01634*, 2022.
12. David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *Nature*, 550(7676):354, 2017.
13. Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphael Marinier, Léonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, et al. What matters in on-policy reinforcement learning? a large-scale empirical study. *arXiv preprint arXiv:2006.05990*, 2020.
14. Richard S Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM SIGART Bulletin*, 2(4):160–163, 1991.

15. Chelsea Finn and Sergey Levine. Deep visual foresight for planning robot motion. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2786–2793. IEEE, 2017.
16. David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.
17. Lukasz Kaiser, Mohammad Babaeizadeh, Piotr Milos, Blazej Osinski, Roy H Campbell, Konrad Czechowski, Dumitru Erhan, Chelsea Finn, Piotr Kozakowski, Sergey Levine, et al. Model-based reinforcement learning for atari. *arXiv preprint arXiv:1903.00374*, 2019.
18. William H Guss, Cayden Codel, Katja Hofmann, Brandon Houghton, Noboru Kuno, Stephanie Milani, Sharada Mohanty, Diego Perez Liebana, Ruslan Salakhutdinov, Nicholay Topin, et al. The minerl competition on sample efficient reinforcement learning using human priors. *arXiv e-prints*, pages arXiv–1904, 2019.
19. Ingmar Kanitscheider, Joost Huizinga, David Farhi, William Hebgen Guss, Brandon Houghton, Raul Sampedro, Peter Zhokhov, Bowen Baker, Adrien Ecoffet, Jie Tang, et al. Multi-task curriculum learning in a complex, visual, hard-exploration domain: Minecraft. *arXiv preprint arXiv:2106.14876*, 2021.
20. Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining (vpt): Learning to act by watching unlabeled online videos. *arXiv preprint arXiv:2206.11795*, 2022.
21. Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019.
22. Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. *arXiv preprint arXiv:2010.02193*, 2020.
23. Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
24. Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. *arXiv preprint arXiv:1811.04551*, 2018.
25. Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
26. Durk P Kingma, Tim Salimans, Rafal Jozefowicz, Xi Chen, Ilya Sutskever, and Max Welling. Improved variational inference with inverse autoregressive flow. *Advances in neural information processing systems*, 29, 2016.
27. Rewon Child. Very deep vaes generalize autoregressive models and can outperform them on images. *arXiv preprint arXiv:2011.10650*, 2020.
28. Marc G Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. In *International Conference on Machine Learning*, pages 449–458. PMLR, 2017.
29. Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
30. Ronald J Williams and Jing Peng. Function optimization using connectionist reinforcement learning algorithms. *Connection Science*, 3(3):241–268, 1991.

31. Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3-4):229–256, 1992.
32. Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *arXiv preprint arXiv:1801.01290*, 2018.
33. Abbas Abdolmaleki, Jost Tobias Springenberg, Yuval Tassa, Remi Munos, Nicolas Heess, and Martin Riedmiller. Maximum a posteriori policy optimisation. *arXiv preprint arXiv:1806.06920*, 2018.
34. J Beau W Webber. A bi-symmetric log transformation for wide-range data. *Measurement Science and Technology*, 24(2):027001, 2012.
35. Steven Kapturowski, Georg Ostrovski, John Quan, Remi Munos, and Will Dabney. Recurrent experience replay in distributed reinforcement learning. In *International conference on learning representations*, 2018.
36. Matteo Hessel, Hubert Soyer, Lasse Espeholt, Wojciech Czarnecki, Simon Schmitt, and Hado van Hasselt. Multi-task deep reinforcement learning with popart. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 3796–3803, 2019.
37. Karl W Cobbe, Jacob Hilton, Oleg Klimov, and John Schulman. Phasic policy gradient. In *International Conference on Machine Learning*, pages 2020–2027. PMLR, 2021.
38. Marc G Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013.
39. Marlos C Machado, Marc G Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018.
40. Matteo Hessel, Joseph Modayil, Hado Van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *Thirty-Second AAAI Conference on Artificial Intelligence*, 2018.
41. Will Dabney, Georg Ostrovski, David Silver, and Rémi Munos. Implicit quantile networks for distributional reinforcement learning. In *International conference on machine learning*, pages 1096–1105. PMLR, 2018.
42. Karl Cobbe, Chris Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International conference on machine learning*, pages 2048–2056. PMLR, 2020.
43. Charles Beattie, Joel Z Leibo, Denis Teplyashin, Tom Ward, Marcus Wainwright, Heinrich Küttler, Andrew Lefrancq, Simon Green, Víctor Valdés, Amir Sadik, et al. Deepmind lab. *arXiv preprint arXiv:1612.03801*, 2016.
44. Weirui Ye, Shaohuai Liu, Thanard Kurutach, Pieter Abbeel, and Yang Gao. Mastering atari games with limited data. *Advances in Neural Information Processing Systems*, 34:25476–25488, 2021.
45. Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample efficient world models. *arXiv preprint arXiv:2209.00588*, 2022.

46. Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, et al. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018.
47. Denis Yarats, Rob Fergus, Alessandro Lazaric, and Lerrel Pinto. Mastering visual continuous control: Improved data-augmented reinforcement learning. *arXiv preprint arXiv:2107.09645*, 2021.
48. Ian Osband, Yotam Doron, Matteo Hessel, John Aslanides, Eren Sezener, Andre Saraiva, Katrina McKinney, Tor Lattimore, Csaba Szepesvari, Satinder Singh, et al. Behaviour suite for reinforcement learning. *arXiv preprint arXiv:1908.03568*, 2019.
49. Olivia Dizon-Paradis, Stephen Wormald, Daniel Capecci, Avanti Bhandarkar, and Damon Woodard. Investigating the practicality of existing reinforcement learning algorithms: A performance comparison. *Authorea Preprints*, 2023.
50. Danijar Hafner. Benchmarking the spectrum of agent capabilities. *arXiv preprint arXiv:2109.06780*, 2021.
51. Denis Yarats, Amy Zhang, Ilya Kostrikov, Brandon Amos, Joelle Pineau, and Rob Fergus. Improving sample efficiency in model-free reinforcement learning from images. *arXiv preprint arXiv:1910.01741*, 2019.
52. Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gomez Colmenarejo, Alexander Novikov, Gabriel Barth-Maron, Mai Gimenez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, et al. A generalist agent. *arXiv preprint arXiv:2205.06175*, 2022.
53. Matthew Johnson, Katja Hofmann, Tim Hutton, and David Bignell. The malmo platform for artificial intelligence experimentation. In *IJCAI*, pages 4246–4247. Citeseer, 2016.
54. Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
55. Shengyi Huang, Rousslan Fernand Julien Dossa, Antonin Raffin, Anssi Kanervisto, and Weixun Wang. The 37 implementation details of proximal policy optimization. *The ICLR Blog Track 2023*, 2022.
56. Matt Hoffman, Bobak Shahriari, John Aslanides, Gabriel Barth-Maron, Feryal Behbahani, Tamara Norman, Abbas Abdolmaleki, Albin Cassirer, Fan Yang, Kate Baumli, et al. Acme: A research framework for distributed reinforcement learning. *arXiv preprint arXiv:2006.00979*, 2020.
57. Simon Schmitt, Matteo Hessel, and Karen Simonyan. Off-policy actor-critic with shared experience replay. In *International Conference on Machine Learning*, pages 8545–8554. PMLR, 2020.
58. Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. *arXiv preprint arXiv:1511.05952*, 2015.
59. Andy Brock, Soham De, Samuel L Smith, and Karen Simonyan. High-performance large-scale image recognition without normalization. In *International Conference on Machine Learning*, pages 1059–1071. PMLR, 2021.
60. Liu Ziyin, Zhikang T Wang, and Masahito Ueda. Laprop: Separating momentum and adaptivity in adam. *arXiv preprint arXiv:2002.04839*, 2020.

61. Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
62. Audrunas Gruslys, Will Dabney, Mohammad Gheshlaghi Azar, Bilal Piot, Marc Bellemare, and Remi Munos. The reactor: A fast and sample-efficient actor-critic agent for reinforcement learning. *arXiv preprint arXiv:1704.04651*, 2017.
63. Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.
64. Matthijs Van Keirsbilck, Alexander Keller, and Xiaodong Yang. Rethinking full connectivity in recurrent neural networks. *arXiv preprint arXiv:1905.12340*, 2019.
65. Marlos C Machado, Marc G Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018.
66. Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Volodymir Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, et al. Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures. *arXiv preprint arXiv:1802.01561*, 2018.

Methods

Baselines

We employ the Proximal Policy Optimization (PPO) algorithm[5], which has become a standard choice in the field, to compare Dreamer under fixed hyperparameters across all benchmarks. There are a large number of PPO implementations available publicly and they are known to substantially vary in task performance[55]. To ensure a comparison that is representative of the highest performance PPO can achieve under fixed hyperparameters across domains, we choose the high-quality PPO implementation available in the Acme framework[56] and select its hyperparameters in Table 1 following recommendations[55,13] and additionally tune its epoch batch size to be large enough for complex environments[42], its learning rate, and its entropy scale. We match the discount factor to Dreamer because it works well across domains and is a common choice in the literature[35,8]. We choose the IMPALA network architecture that we have found performed better than alternatives[42] and set the minibatch size to the largest possible for one A100 GPU. We verify the performance of our PPO implementation and hyperparameters on the ProcGen benchmark, where a highly tuned PPO implementation has been reported by the PPO authors[37]. We find that our implementation matches or slightly outperforms this performance reference.

Parameter	**Value**
Observation normalization	Yes
Reward normalization	Yes
Reward clipping (stddev.)	10
Epoch batch	64×256
Number of epochs	3
Minibatch size	8
Minibatch length	256
Policy trust region	0.2
Value trust region	No
Advantage normalization	Yes
Entropy penalty scale	0.01
Discount factor	0.997
GAE lambda	0.95
Learning rate	3×10^{-4}
Gradient clipping	0.5
Adam epsilon	10^{-5}

Table 1: PPO hyperparameters used across all benchmarks.

For Minecraft, we additionally tune and run the IMPALA and Rainbow algorithms because not successful end-to-end learning from scratch has been reported in the literature[18]. We use the Acme implementations[56] of these algorithms, use the same IMPALA network we used for PPO, and tuned the learning rate and entropy regularizers. For all other benchmarks, we compare to tuned expert algorithms reported in the literature as referenced in the results section.

Implementation

Experience replay We implement Dreamer using a uniform replay buffer with an online queue[57]. Specifically, each minibatch is formed first from non-overlapping online trajectories and then filled up with uniformly sampled trajectories from the replay buffer. We store latent states into the replay buffer during data collection to initialize the world model on replayed trajectories, and write the fresh latent states of the training rollout back into the buffer. While prioritized replay[58] is used by some of the expert algorithms we compare to and we found it to also improve the performance of Dreamer, we opt for uniform replay in our experiments for ease of implementation.

We parameterize the amount of training via the replay ratio. This is the fraction of time steps trained on per time step collected from the environment, without action repeat. Dividing the replay ratio by the time steps in a minibatch and by action repeat yields the ratio of gradient steps to env steps. For example, a replay ratio of 32 on Atari with action repeat of 4 and batch shape 16×64 corresponds to 1 gradient step every 128 env steps, or 1.5M gradient steps over 200M env steps.

Optimizer We employ Adaptive Gradient Clipping (AGC)[59], which clips per-tensor gradients if they exceed 30% of the L2 norm of the weight matrix they correspond to, with its default $\epsilon = 10^{-3}$. AGC decouples the clipping threshold from the loss scales, allowing to change loss functions or loss scales without adjusting the clipping threshold. We apply the clipped gradients using the LaProp optimizer[60] with $\epsilon = 10^{-20}$ and its default parameters $\beta_1 = 0.9$ and $\beta_2 = 0.99$. LaProp normalizes gradients by RMSProp and then smoothes them by momentum, instead of computing both momentum and normalizer on raw gradients as Adam does[61]. This simple change allows for a smaller epsilon and avoids occasional instabilities that we observed under Adam.

Distributions The encoder, dynamics predictor, and actor distributions are mixtures of 99% the predicted softmax output and 1% of a uniform distribution[62] to prevent zero probabilities and infinite log probabilities. The rewards and critic neural networks output a softmax distribution over exponentially spaced bins $b \in B$ and are trained towards twohot encoded targets:

$$\operatorname{twohot}(x)_i \doteq \begin{cases} |b_{k+1} - x| \,/\, |b_{k+1} - b_k| & \text{if } i = k \\ |b_k \quad - x| \,/\, |b_{k+1} - b_k| & \text{if } i = k+1 \\ 0 & \text{else} \end{cases} \qquad k \doteq \sum_{j=1}^{|B|} \delta(b_j < x) \tag{12}$$

The output weights of twohot distributions are initialized to zero to ensure that the agent does not hallucinate rewards and values at initialization. For computing the expected prediction of the softmax distribution under bins that span many orders of magnitude, the summation order matters and positive and negative bins should be summed up separately, from small to large bins, and then added. Refer to the source code for an implementation.

Networks Images are encoded using stride 2 convolutions to resolution 6×6 or 4×4 and then flattened and are decoded using transposed stride 2 convolutions, with sigmoid activation on the output. Vector inputs are symlog transformed and then encoded and decoded using 3-layer MLPs. The actor and critic neural networks are also 3-layer MLPs and the reward and continue predictors are 1-layer MLPs. The sequence model is a GRU[63] with block-diagonal recurrent weights[64] of 8 blocks to allow for a large number of memory units without quadratic increase in parameters and FLOPs. The input to the GRU at each time step is a linear embedding of the sampled latent z_t, of the action a_t, and of the recurrent state to allow mixing between blocks.

Benchmarks

Protocols Summarized in Table 2, we follow the standard evaluation protocols for the benchmarks where established. Atari[38] uses 57 tasks with sticky actions[65]. The random and human reference scores used to normalize scores vary across the literature and we chose the most common reference values, replicated in Table 6. DMLab[43] uses 30 tasks[66] and we use the fixed action space[36,35]. We evaluate at 100M steps because running for 10B as in some prior work was infeasible. Because existing published baselines perform poorly at 100M steps, we compare to their performance at 1B steps instead, giving them a $10\times$ data advantage. ProcGen uses the hard difficulty setting and the unlimited level set[42]. Prior work compares at different step budgets[42,37] and we compare at 50M steps due to computational cost, as there is no action repeat. For Minecraft Diamond purely from sparse rewards, we establish the evaluation protocol to report the episode return measured at 100M env steps, corresponding to about 100 days of in-game time. Atari100k[17] includes 26 tasks with a budget of 400K env steps, 100K after action repeat. Prior work has used various environment settings, summarized in Table 10, and we chose the environments as originally introduced. Visual Control[46,21] spans 20 tasks with an action repeat of 2. Proprioceptive Control follows the same protocol but we exclude the two quadruped tasks because of baseline availability in prior work[47].

Benchmark	Tasks	Env Steps	Action Repeat	Env Instances	Replay Ratio	GPU Days	Model Size
Minecraft	1	100M	1	64	32	8.9	200M
DMLab	30	100M	4	16	32	2.9	200M
ProcGen	16	50M	1	16	64	16.1	200M
Atari	57	200M	4	16	32	7.7	200M
Atari100K	26	400K	4	1	128	0.1	200M
BSuite	23	—	1	1	1024	0.5	200M
Proprio Control	18	500K	2	16	512	0.3	12M
Visual Control	20	1M	2	16	512	0.1	12M

Table 2: Benchmark overview. All agents were trained on a single Nvidia A100 GPU each.

Environment instances In earlier experiments, we observed that the performance of both Dreamer and PPO is robust to the number of environment instances. Based on the CPU resources available on our training machines, we use 16 environment instance by default. For BSuite, the benchmark requires using a single environment instance. We also use a single environment instance for Atari100K because the benchmark has a budget of 400K env steps whereas the maximum episode length in Atari is in principle 432K env steps. For Minecraft, we use 64 environments using remote CPU workers to speed up experiments because the environment is slower to step.

Seeds and error bars We run 5 seeds for each Dreamer and PPO per benchmark, with the exception of 1 seed for ProcGen due to computational constraints, 10 seeds for BSuite as required by the benchmark, and 10 seeds for Minecraft to reliably report the fraction of runs that achieve diamonds. All curves show the mean over seeds with one standard deviation shaded.

Computational choices All Dreamer and PPO agents in this paper were trained on a single Nvidia A100 GPU each. Dreamer uses the 200M model size by default. On the two control suitse, Dreamer the same performance using the substantially faster 12M model, making it more accessible to researchers. The replay ratio control the trade-off between computational cost and data efficiency as analyzed in Figure 6 and is chosen to fit the step budget of each benchmark.

Model sizes

To accommodate different computational budgets and analyze robustness to different model sizes, we define a range of models ranging from 12M to 400M parameters shown in Table 3. The sizes are parameterized by the model dimension, which approximately increases in multiples of 1.5, alternating between powers of two and power of two scaled by 1.5. This yields tensor shapes that are multiples of 8 as required for hardware efficiency. Sizes of different network components derive from the model dimension. The MLPs have the model dimension as the number of hidden units. The sequence model has 8 times the number of recurrent units, split into 8 blocks of the same size as the MLPs. The convolutional encoder and decoder layers closest to the data use $16\times$ fewer channels than the model dimension. Each latent also uses $16\times$ fewer codes than the model dimension. The number of hidden layers and number of latents is fixed across model sizes. All hyperparamters, including the learning rate and batch size, are fixed across model sizes.

Parameters	**12M**	**25M**	**50M**	**100M**	**200M**	**400M**
Hidden size (d)	256	384	512	768	1024	1536
Recurrent units ($8d$)	1024	3072	4096	6144	8192	12288
Base CNN channels ($d/16$)	16	24	32	48	64	96
Codes per latent ($d/16$)	16	24	32	48	64	96

Table 3: Dreamer model sizes. The number of MLP hidden units defines the model dimension, from which recurrent units, convolutional channels, and number of codes per latent are derived. The number of layers and latents is constant across model sizes.

Previous Dreamer generations

We present the third generation of the Dreamer line of work. Where the distinction is useful, we refer to this algorithm as DreamerV3. The DreamerV1 algorithm[21] was limited to continuous control, the DreamerV2 algorithm[22] surpassed human performance on Atari, and the DreamerV3 algorithm enables out-of-the-box learning across diverse benchmarks.

We summarize the changes that DreamerV3 introduces as follows:

- Robustness techniques: Observation symlog, KL balance and free bits, 1% unimix for all categoricals, percentile return normalization, symexp twohot loss for the reward head and critic.
- Network architecture: Block GRU, RMSNorm normalization, SiLu activation.
- Optimizer: Adaptive gradient clipping (AGC), LaProp (RMSProp followed by momentum).
- Replay buffer: Larger capacity, online queue, storing and updating latent states.

Hyperparameters

Name	Symbol	Value
General		
Replay capacity	—	5×10^6
Batch size	B	16
Batch length	T	64
Activation	—	$\mathrm{RMSNorm} + \mathrm{SiLU}$
Learning rate	—	4×10^{-5}
Gradient clipping	—	$\mathrm{AGC}(0.3)$
Optimizer	—	$\mathrm{LaProp}(\epsilon = 10^{-20})$
World Model		
Reconstruction loss scale	β_{pred}	1
Dynamics loss scale	β_{dyn}	1
Representation loss scale	β_{rep}	0.1
Latent unimix	—	1%
Free nats	—	1
Actor Critic		
Imagination horizon	H	15
Discount horizon	$1/(1-\gamma)$	333
Return lambda	λ	0.95
Critic loss scale	β_{val}	1
Critic replay loss scale	β_{repval}	0.3
Critic EMA regularizer	—	1
Critic EMA decay	—	0.98
Actor loss scale	β_{pol}	1
Actor entropy regularizer	η	3×10^{-4}
Actor unimix	—	1%
Actor RetNorm scale	S	$\mathrm{Per}(R, 95) - \mathrm{Per}(R, 5)$
Actor RetNorm limit	L	1
Actor RetNorm decay	—	0.99

Table 4: Dreamer hyperparameters. The same values are used across all benchmarks, including proprioceptive and visual inputs, continuous and discrete actions, and 2D and 3D domains. We do not use any hyperparameter annealing, prioritized replay, weight decay, or dropout.

Minecraft

Game description With 100M monthly active users, Minecraft is one of the most popular video games worldwide. Minecraft features a procedurally generated 3D world of different biomes, including plains, forests, jungles, mountains, deserts, taiga, snowy tundra, ice spikes, swamps, savannahs, badlands, beaches, stone shores, rivers, and oceans. The world consists of 1 meter sized blocks that the player and break and place. There are about 30 different creatures that the player can interact with or fight. From gathered resources, the player can use over 350 recipes to craft new items and progress through the technology tree, all while ensuring safety and food supply to survive. There are many conceivable tasks in Minecraft and as a first step, the research community has focused on the salient task of obtaining a diamonds, a rare item found deep underground and requires progressing through the technology tree.

Learning environment We built the Minecraft Diamond environment on top of MineRL to define a flat categorical action space and fix issues we discovered with the original environments via human play testing. For example, when breaking diamond ore, the item sometimes jumps into the inventory and sometimes needs to be collected from the ground. The original environment terminates episodes when breaking diamond ore so that many successful episodes end before collecting the item and thus without the reward. We remove this early termination condition and end episodes when the player dies or after 36000 steps, corresponding to 30 minutes at the control frequency of 20Hz. Another issue is that the jump action has to be held for longer than one control step to trigger a jump, which we solve by keeping the key pressed in the background for 200ms. We built the environment on top of MineRL v0.4.4[18], which offers abstract crafting actions. The Minecraft version is 1.11.2.

Observations and rewards The agent observes a $64 \times 64 \times 3$ first-person image, an inventory count vector for the over 400 items, a vector of maximum inventory counts since episode begin to tell the agent which milestones it has achieved, a one-hot vector indicating the equipped item, and scalar inputs for the health, hunger, and breath levels. We follow the sparse reward structure of the MineRL competition environment[18] that rewards 12 milestones leading up to the diamond, for obtaining the items log, plank, stick, crafting table, wooden pickaxe, cobblestone, stone pickaxe, iron ore, furnace, iron ingot, iron pickaxe, and diamond. The reward for each item is only given once per episode, and the agent has to learn to collect certain items multiple times to achieve the next milestone. To make the return easy to interpret, we give a reward of $+1$ for each milestone instead of scaling rewards based on how valuable each item is. Additionally, we give -0.01 for each lost heart and $+0.01$ for each restored heart, but did not investigate whether this is helpful.

Supplementary material

Minecraft video predictions

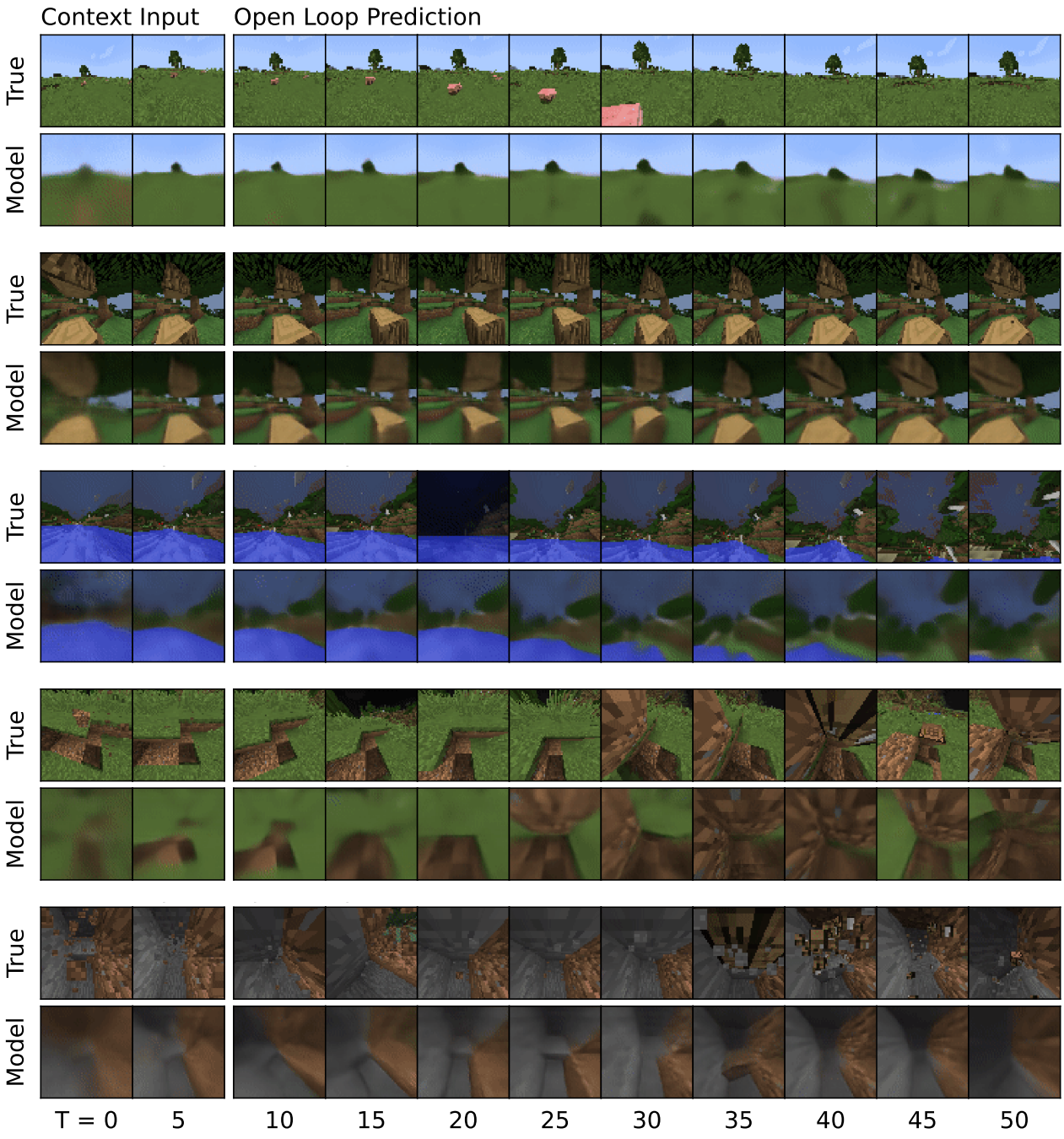


Figure 7: Multi-step predictions on Minecraft. The world model receives the first 5 frames as context input and the predicts 45 steps into the future given the action sequence and without access to intermediate images.

Minecraft additional results

Method	Return
Dreamer	9.1
IMPALA	7.1
Rainbow	6.3
PPO	5.1

Table 5: Minecraft Diamond scores at 100M environment steps.

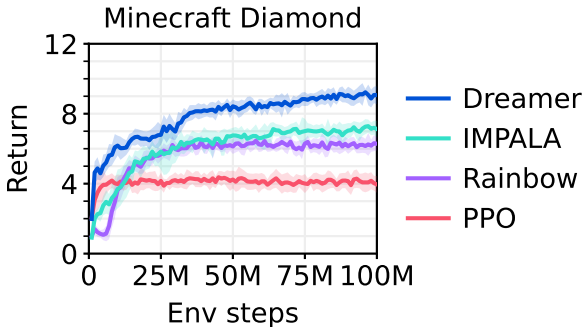


Figure 8: Minecraft learning curves.

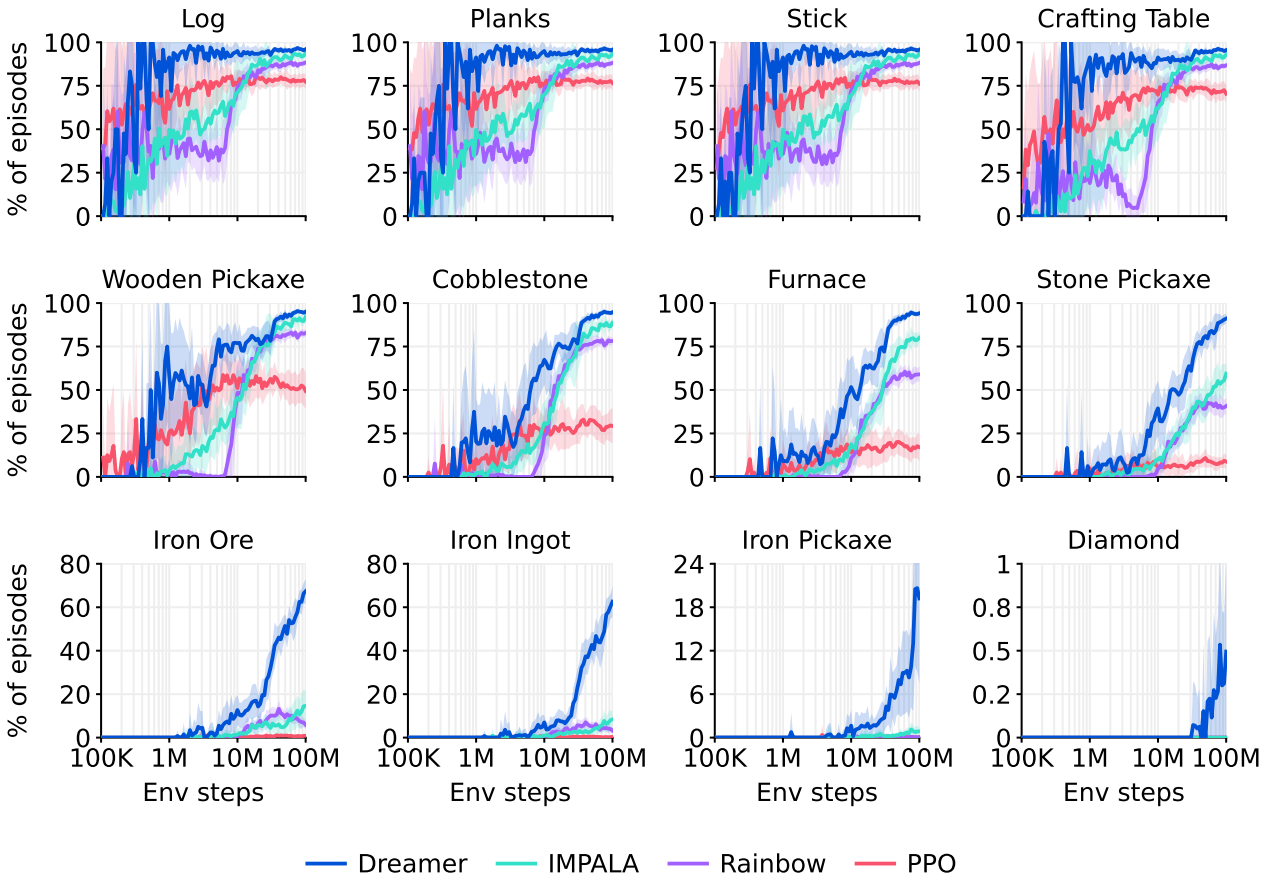


Figure 9: Item success rates as a percentage of episodes. Dreamer obtains items at substantially higher rates than the baselines and continues to improve until the 100M step budget. At the budget, Dreamer obtains diamonds in 0.4% of episodes, leaving a challenge for future research. This metric differs from Figure 5, which shows that over the course of training, 100% of Dreamer agents obtain one or more diamonds regardless of episode boundaries, compared to 0% of the baseline agents.

Atari learning curves

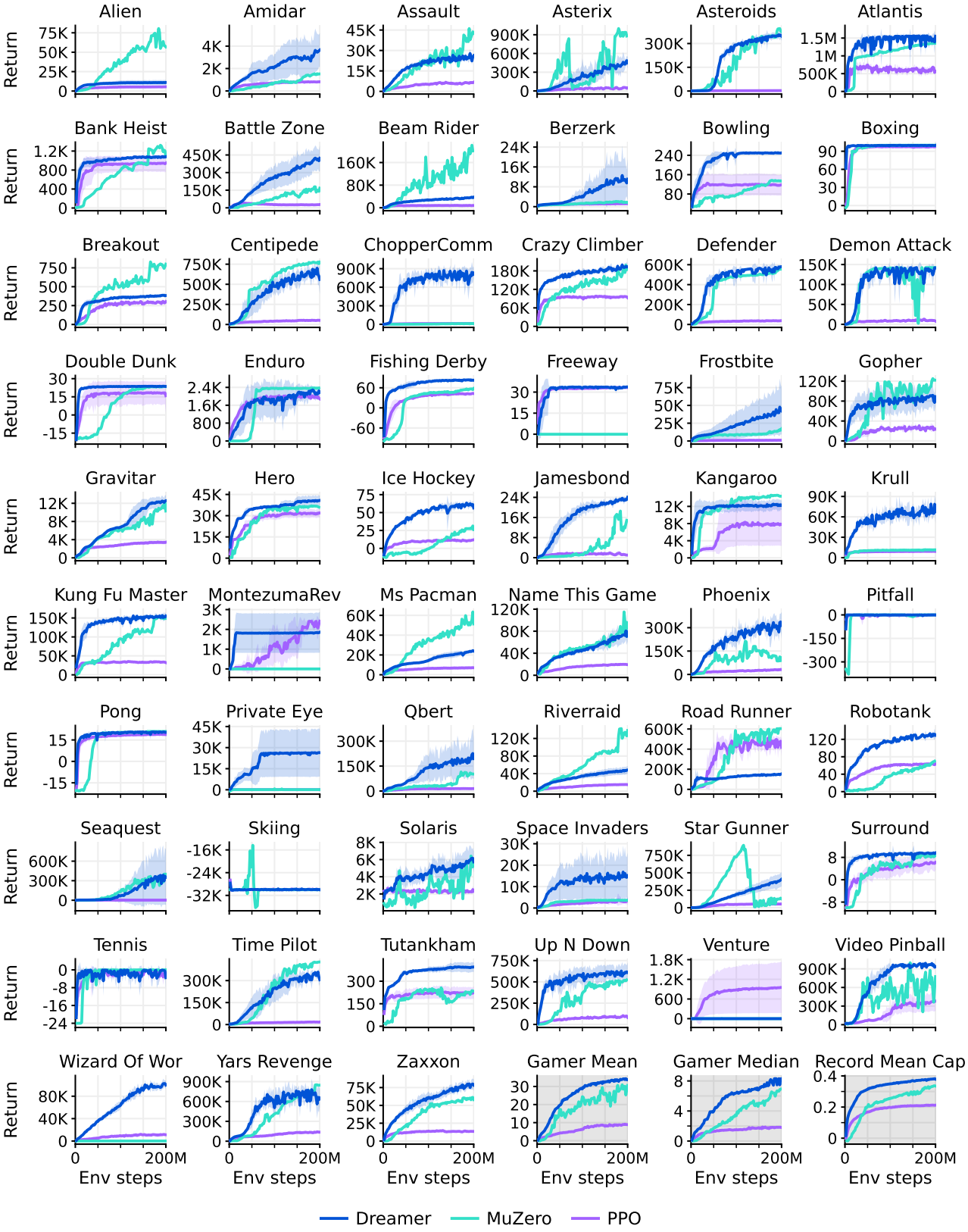


Figure 10: Atari learning curves.

Atari scores

Task	Random	Gamer	Record	PPO	MuZero	Dreamer
Environment steps	—	—	—	200M	200M	200M
Alien	228	7128	251916	5476	**56835**	10977
Amidar	6	1720	104159	817	1517	**3612**
Assault	222	742	8647	6673	**42742**	26010
Asterix	210	8503	1000000	47190	**879375**	441763
Asteroids	719	47389	10506650	2479	**374146**	348684
Atlantis	12850	29028	10604840	539721	1353617	**1553222**
Bank Heist	14	753	82058	946	**1077**	**1083**
Battle Zone	2360	37188	801000	27816	167412	**419653**
Beam Rider	364	16926	999999	7973	**201154**	37073
Berzerk	124	2630	1057940	1186	1698	**10557**
Bowling	23	161	300	118	133	**250**
Boxing	0	12	100	**98**	**100**	**100**
Breakout	2	30	864	299	**799**	384
Centipede	2091	12017	1301709	51833	**774421**	554553
Chopper Command	811	7388	999999	12667	8945	**802698**
Crazy Climber	10780	35829	219900	93176	**184394**	**193204**
Defender	2874	18689	6010500	38270	**554492**	**579875**
Demon Attack	152	1971	1556345	8229	**142509**	**142109**
Double Dunk	–19	–16	22	16	**23**	**24**
Enduro	0	860	9500	1887	**2369**	2166
Fishing Derby	–92	–39	71	43	58	**82**
Freeway	0	30	38	**33**	0	**34**
Frostbite	65	4335	454830	1123	17087	**41888**
Gopher	258	2412	355040	24792	**122025**	87600
Gravitar	173	3351	162850	3436	10301	**12570**
Hero	1027	30826	1000000	31967	36063	**40677**
Ice Hockey	–11	1	36	12	26	**57**
Jamesbond	29	303	45550	1019	14872	**24010**
Kangaroo	52	3035	1424600	7769	**14380**	12229
Krull	1598	2666	104100	9193	11476	**69858**
Kung Fu Master	258	22736	1000000	32335	**148936**	**154893**
Montezuma Revenge	0	4753	1219200	**2368**	0	1852
Ms Pacman	307	6952	290090	7041	**51310**	24079
Name This Game	2292	8049	25220	19441	**85331**	77809
Phoenix	761	7243	4014440	31412	105593	**316606**
Pitfall	–229	6464	114000	–2	**0**	**0**
Pong	–21	15	21	19	**21**	**20**
Private Eye	25	69571	101800	73	100	**26432**
Qbert	164	13455	2400000	14554	102129	**201084**
Riverraid	1338	17118	1000000	14860	**137983**	48080
Road Runner	12	7845	2038100	423995	**604083**	150402
Robotank	2	12	76	63	70	**132**
Seaquest	68	42055	999999	1927	**399764**	356584
Skiing	–17098	–4337	–3272	**–29926**	**–30000**	**–29965**
Solaris	1236	12327	111420	2368	**5860**	**5851**
Space Invaders	148	1669	621535	3489	3639	**15005**
Star Gunner	664	10250	77400	53439	127417	**408961**
Surround	–10	6	6	6	**9**	**9**
Tennis	–24	–8	21	–1	**0**	–3
Time Pilot	3568	5229	65300	17250	**427209**	314947
Tutankham	11	168	5384	225	235	**395**
Up N Down	533	11693	82840	83743	522962	**614065**
Venture	0	1188	38900	**953**	0	0
Video Pinball	16257	17668	89218328	382306	775304	**940631**
Wizard Of Wor	564	4756	395300	10910	0	**99136**
Yars Revenge	3093	54577	15000105	137164	**846061**	675774
Zaxxon	32	9173	83700	13599	58115	**78443**
Gamer median (%)	0	100	3716	180	693	**830**
Gamer mean (%)	0	100	123001	892	3054	**3381**
Record mean (%)	0	13	100	21	66	**74**
Record mean capped (%)	0	13	100	21	34	**38**

Table 6: Atari scores.

ProcGen learning curves

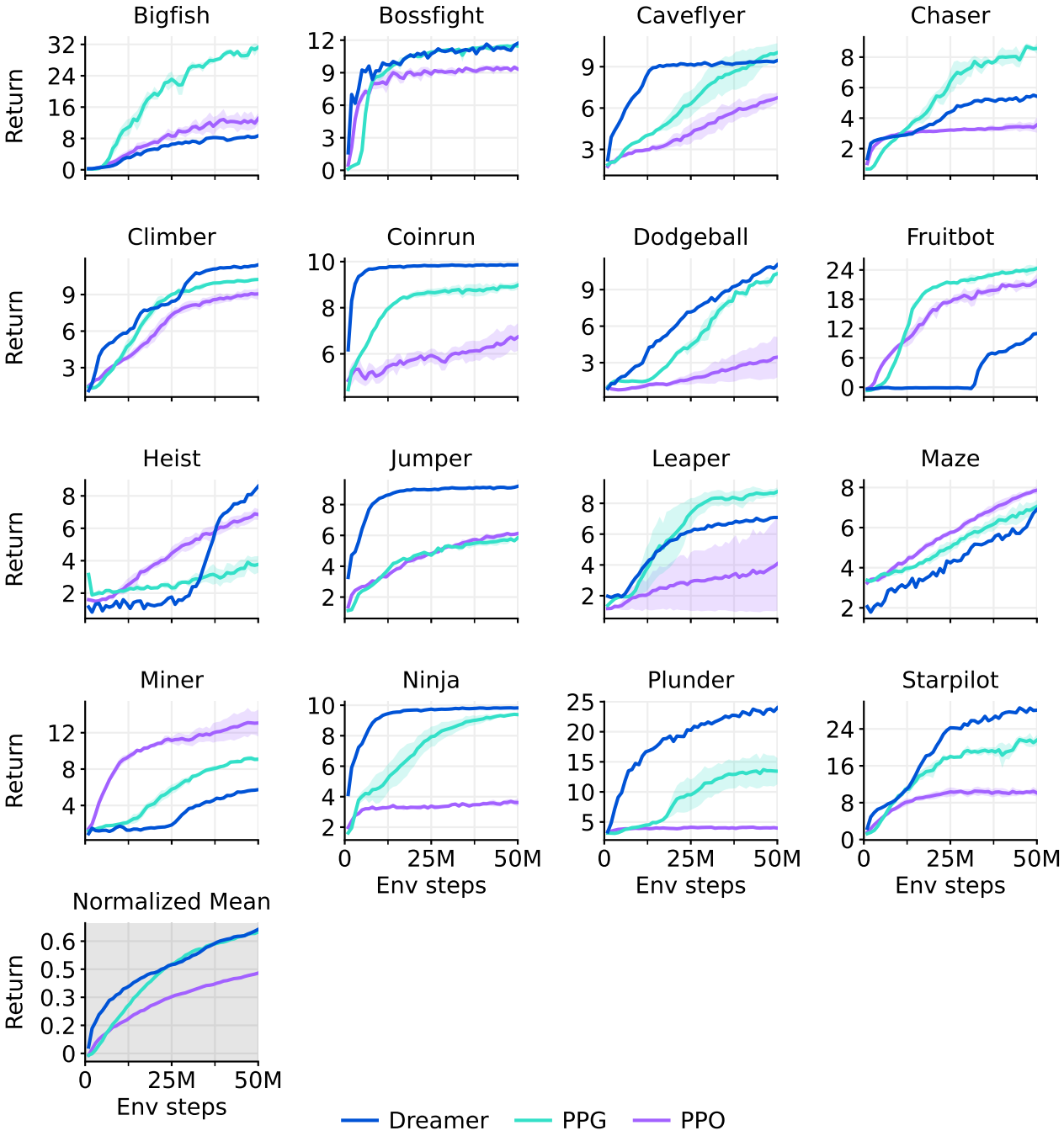


Figure 11: ProcGen learning curves.

ProcGen scores

Task	Original PPO	PPO	PPG	Dreamer
Environment steps	50M	50M	50M	50M
Bigfish	10.92	12.72	**31.26**	8.62
Bossfight	10.47	9.36	**11.46**	**11.61**
Caveflyer	6.03	6.71	**10.02**	9.42
Chaser	4.48	3.54	**8.57**	5.49
Climber	7.59	9.04	10.24	**11.43**
Coinrun	7.93	6.71	8.98	**9.86**
Dodgeball	4.80	3.44	10.31	**10.93**
Fruitbot	20.28	21.69	**24.32**	11.04
Heist	2.25	6.87	3.77	**8.51**
Jumper	5.09	6.13	5.84	**9.17**
Leaper	5.90	4.07	**8.76**	7.05
Maze	4.97	**7.86**	7.06	6.85
Miner	7.56	**12.97**	9.08	5.71
Ninja	6.16	3.62	**9.38**	**9.82**
Plunder	11.16	3.99	13.44	**23.81**
Starpilot	17.00	10.13	21.57	**28.00**
Normalized mean	41.16	42.80	**64.89**	**66.01**

Table 7: ProcGen scores. The PPO implementation we use throughout our paper under fixed hyperparameters performs on par or better than the original PPO, which its authors describe as highly tuned with near optimal hyperparameters[37].

DMLab learning curves

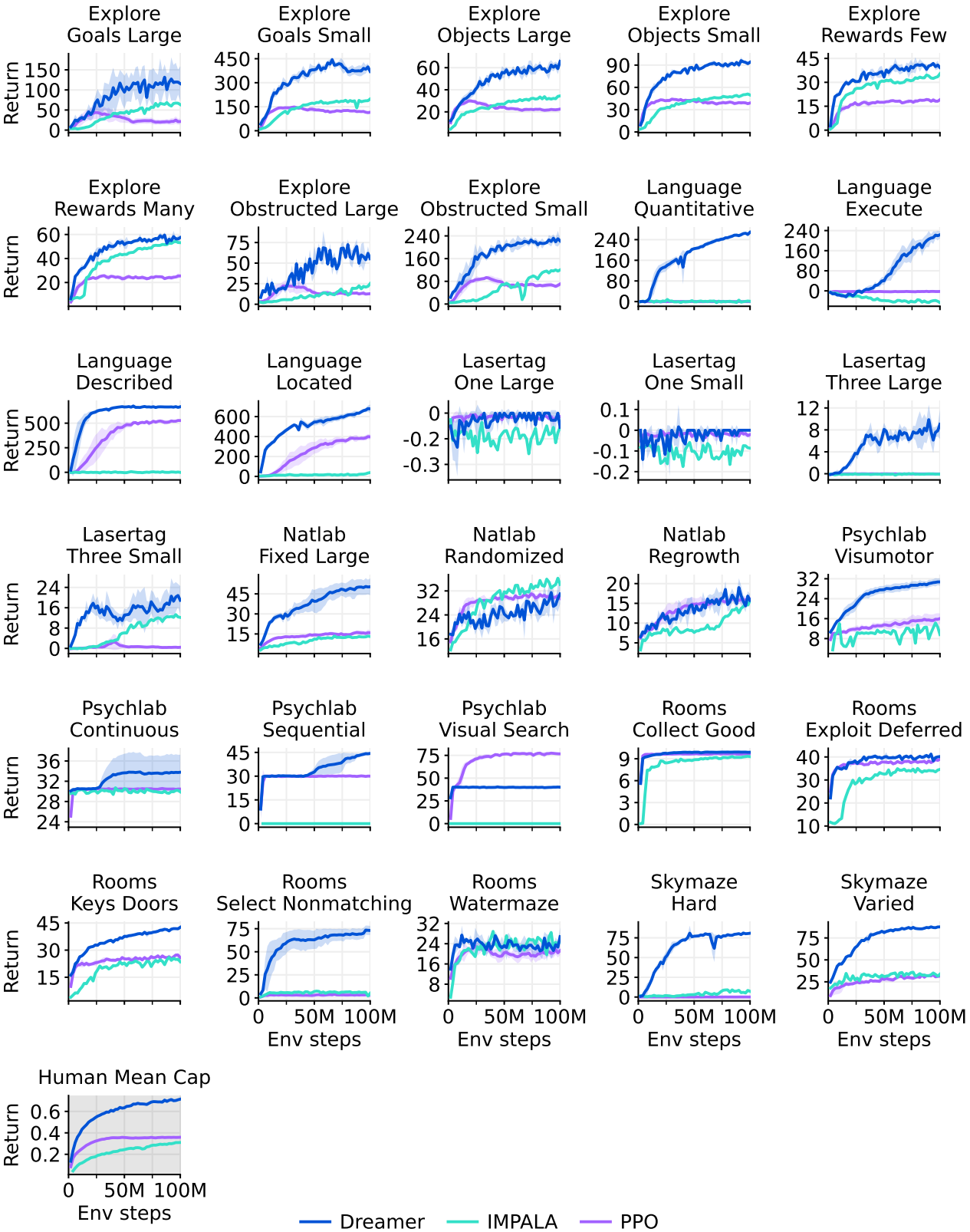


Figure 12: DMLab learning curves.

DMLab scores

Task	R2D2+	IMPALA	IMPALA	IMPALA	PPO	Dreamer
Environment steps	10B	10B	1B	100M	100M	100M
Explore Goal Locations Large	174.7	316.0	137.8	64.2	21.2	**116.7**
Explore Goal Locations Small	460.7	482.0	302.8	196.1	115.1	**372.8**
Explore Object Locations Large	60.6	91.0	55.1	34.3	22.5	**63.9**
Explore Object Locations Small	83.7	100.4	75.9	50.6	38.9	**93.5**
Explore Object Rewards Few	80.7	92.6	46.9	34.6	19.0	**40.0**
Explore Object Rewards Many	75.8	89.4	68.5	53.3	25.5	**58.1**
Explore Obstructed Goals Large	95.5	102.0	57.9	23.7	12.9	**52.8**
Explore Obstructed Goals Small	311.9	372.0	214.9	118.0	70.4	**224.6**
Language Answer Quantitative Question	344.4	362.0	304.7	1.0	0.3	**266.0**
Language Execute Random Task	497.4	465.4	140.8	–44.4	–2.5	**223.7**
Language Select Described Object	617.6	664.0	618.4	0.2	526.9	**665.5**
Language Select Located Object	772.8	731.4	413.0	38.2	397.5	**679.5**
Lasertag One Opponent Large	0.0	0.0	0.0	–0.1	**0.0**	–0.1
Lasertag One Opponent Small	31.8	0.0	0.0	–0.1	**0.0**	**0.0**
Lasertag Three Opponents Large	28.6	32.2	10.4	–0.1	0.0	**9.0**
Lasertag Three Opponents Small	49.0	57.2	37.1	12.1	0.4	**18.8**
Natlab Fixed Large Map	60.6	63.4	53.8	13.0	15.8	**50.5**
Natlab Varying Map Randomized	42.4	47.0	40.5	**35.3**	29.6	31.2
Natlab Varying Map Regrowth	24.6	34.0	25.5	15.3	**16.3**	**16.7**
Psychlab Arbitrary Visuomotor Mapping	33.1	38.4	16.5	11.9	16.0	**30.7**
Psychlab Continuous Recognition	30.0	28.6	30.0	30.1	30.5	**33.8**
Psychlab Sequential Comparison	30.0	29.6	0.0	0.0	30.0	**44.3**
Psychlab Visual Search	79.9	80.0	0.0	0.0	**76.6**	40.1
Rooms Collect Good Objects Test	9.9	10.0	9.9	9.3	**9.7**	**9.9**
Rooms Exploit Deferred Effects Test	38.1	62.2	37.6	34.5	**39.0**	**40.4**
Rooms Keys Doors Puzzle	46.2	54.6	36.9	24.2	26.0	**42.4**
Rooms Select Nonmatching Object	63.6	39.0	63.2	4.0	2.7	**73.1**
Rooms Watermaze	49.0	47.0	50.1	23.6	21.2	**26.1**
Skymaze Irreversible Path Hard	76.0	80.0	46.4	7.7	0.0	**80.9**
Skymaze Irreversible Path Varied	76.0	100.0	69.8	32.7	31.3	**87.7**
Human mean capped (%)	85.4	85.1	66.3	31.0	35.9	**71.4**

Table 8: DMLab scores at 100M environment steps and larger budgets. The IMPALA agent corresponds to "IMPALA (deep)" presented by Kapturowski et al.[35] who made the learning curves available.

Atari100k learning curves

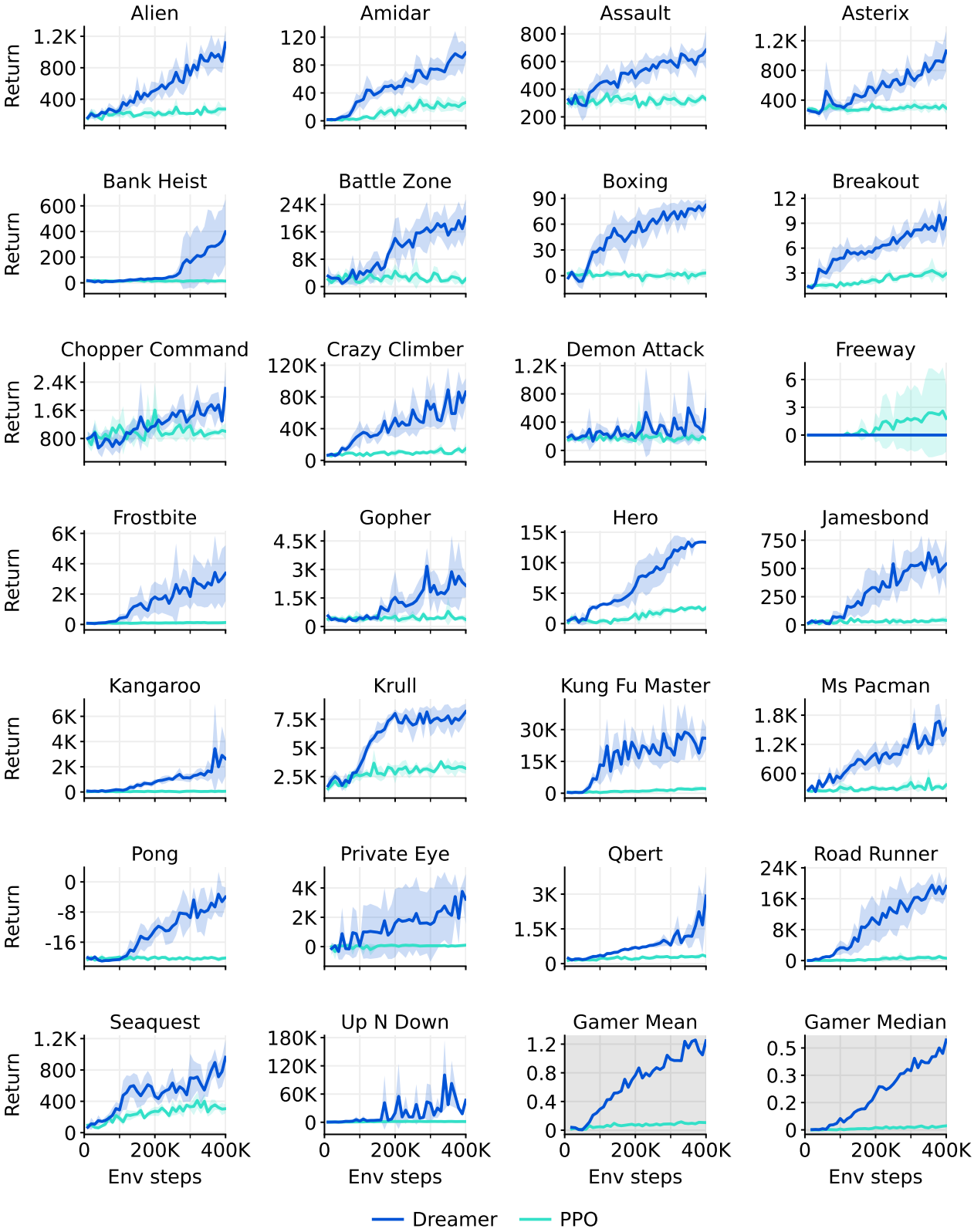


Figure 13: Atari100k learning curves.

Atari100k scores

Task	Random	Human	PPO	SimPLe	SPR	TWM	IRIS	Dreamer
Environment steps	—	—	400K	400K	400K	400K	400K	400K
Alien	228	7128	276	617	842	675	420	**1118**
Amidar	6	1720	26	74	**180**	122	143	97
Assault	222	742	327	527	566	683	**1524**	683
Asterix	210	8503	292	**1128**	962	**1117**	854	1062
Bank Heist	14	753	14	34	345	**467**	53	398
Battle Zone	2360	37188	2233	4031	14834	5068	13074	**20300**
Boxing	0	12	3	8	36	**78**	70	**82**
Breakout	2	30	3	16	20	20	**84**	10
Chopper Command	811	7388	1005	979	946	1697	1565	**2222**
Crazy Climber	10780	35829	14675	62584	36700	71820	59324	**86225**
Demon Attack	152	1971	160	208	518	350	**2034**	577
Freeway	0	30	2	17	19	24	**31**	0
Frostbite	65	4335	127	237	1171	1476	259	**3377**
Gopher	258	2412	368	597	661	1675	**2236**	**2160**
Hero	1027	30826	2596	2657	5859	7254	7037	**13354**
Jamesbond	29	303	41	100	366	362	463	**540**
Kangaroo	52	3035	55	51	**3617**	1240	838	2643
Krull	1598	2666	3222	2205	3682	6349	6616	**8171**
Kung Fu Master	258	22736	2090	14862	14783	24555	21760	**25900**
Ms Pacman	307	6952	366	1480	1318	**1588**	999	**1521**
Pong	–21	15	–20	13	–5	**19**	15	–4
Private Eye	25	69571	100	35	86	87	100	**3238**
Qbert	164	13455	317	1289	866	**3331**	746	2921
Road Runner	12	7845	602	5641	12213	9109	9615	**19230**
Seaquest	68	42055	305	683	558	774	661	**962**
Up N Down	533	11693	1502	3350	10859	15982	3546	**46910**
Gamer mean (%)	0	100	11	33	62	96	105	**125**
Gamer median (%)	0	100	2	13	40	**51**	29	**49**

Table 9: Atari100k scores at 400K environment steps, corresponding to 100k agent steps.

Setting	SimPLe	EffMuZero	SPR	IRIS	TWM	Dreamer
Gamer score (%)	33	190	62	105	96	125
Gamer median (%)	13	109	40	29	51	49
GPU days	5.0	0.6	0.1	3.5	0.4	0.1
Online planning	—	X	—	—	—	—
Data augmentation	—	—	X	—	—	—
Non-uniform replay	—	X	X	—	X	—
Separate hparams	—	—	—	X	—	—
Increased resolution	—	X	X	—	—	—
Uses life information	—	X	X	X	X	—
Uses early resets	—	X	—	X	—	—
Separate eval episodes	X	X	X	X	X	—

Table 10: Evaluation protocols for the Atari 100k benchmark. Computational resources are converted to A100 GPU days. EfficientMuZero[44] achieves the highest scores but changed the environment configuration from the standard[17]. IRIS uses a separate hyperparameter for its exploration strength on Freeway.

Proprioceptive control learning curves

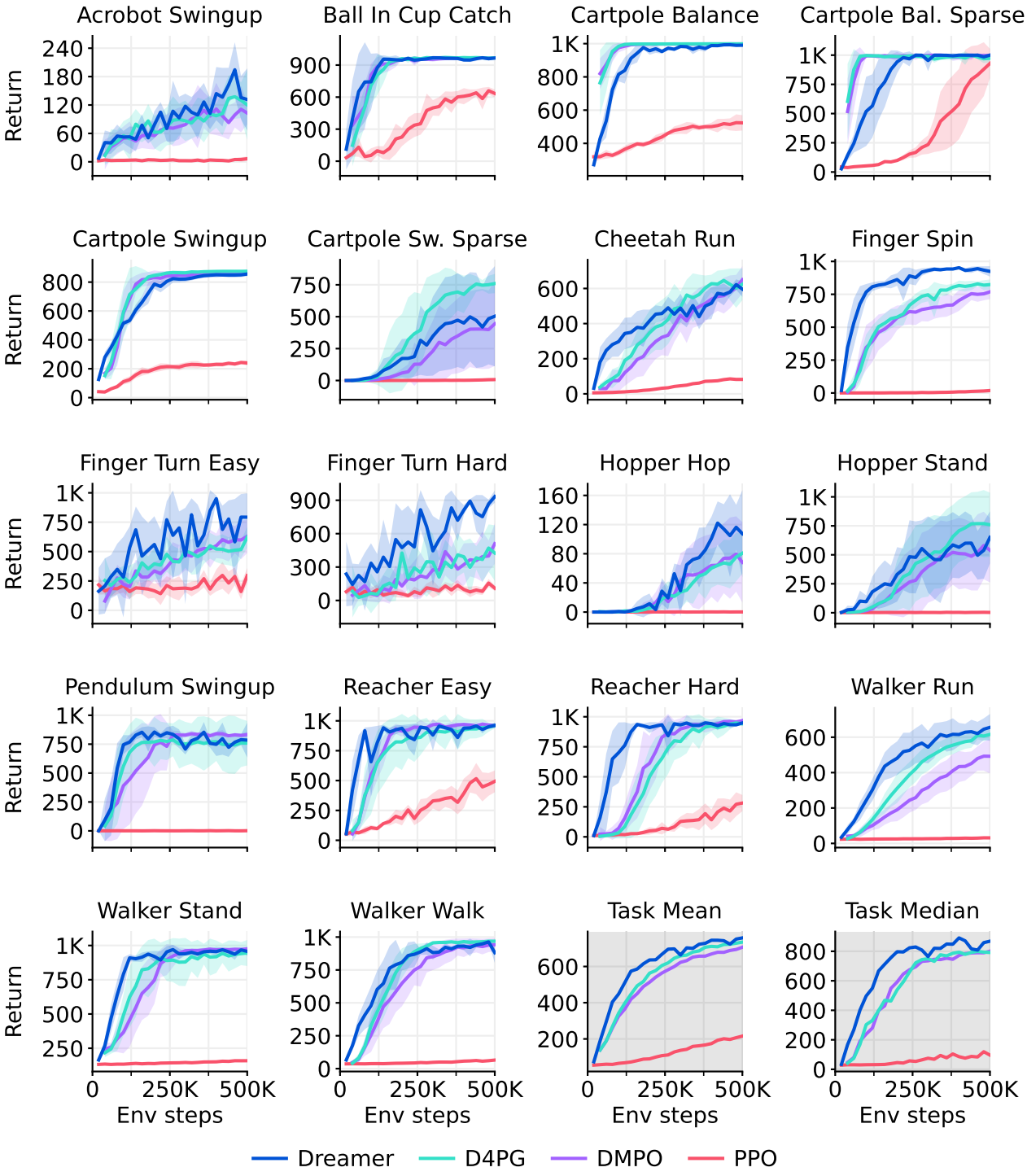


Figure 14: DeepMind Control Suite learning curves under proprioceptive inputs.

Proprioceptive control scores

Task	PPO	DDPG	DMPO	D4PG	Dreamer
Environment steps	500K	500K	500K	500K	500K
Acrobot Swingup	6	100	103	124	**134**
Ball In Cup Catch	632	917	**968**	**968**	**962**
Cartpole Balance	523	**997**	**999**	**999**	**990**
Cartpole Balance Sparse	930	**992**	**999**	**974**	**990**
Cartpole Swingup	240	**864**	**860**	**875**	**852**
Cartpole Swingup Sparse	7	703	438	**752**	491
Cheetah Run	82	596	**650**	**624**	614
Finger Spin	18	775	769	823	**931**
Finger Turn Easy	281	499	620	612	**793**
Finger Turn Hard	106	313	495	421	**889**
Hopper Hop	0	36	68	80	**113**
Hopper Stand	3	484	549	**762**	576
Pendulum Swingup	1	767	**834**	759	788
Reacher Easy	494	**934**	**961**	**960**	**954**
Reacher Hard	288	**949**	**968**	**937**	**938**
Walker Run	31	561	493	616	**649**
Walker Stand	159	**965**	**975**	**947**	**964**
Walker Walk	64	**952**	**942**	**969**	**936**
Task mean	94	771	801	792	**871**
Task median	215	689	705	**733**	**754**

Table 11: DeepMind Control Suite scores under proprioceptive inputs.

Visual control learning curves

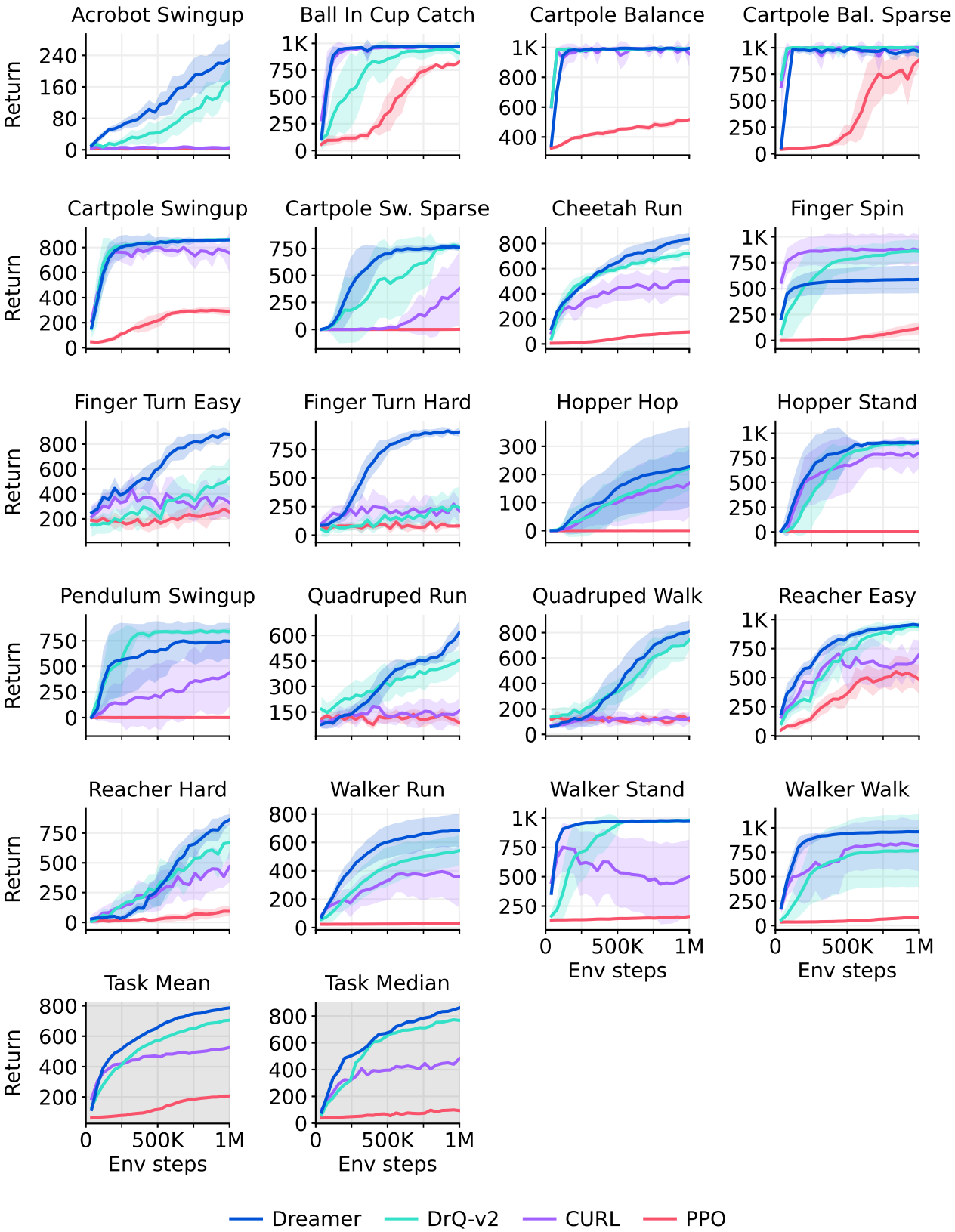


Figure 15: DeepMind Control Suite learning curves under visual inputs.

Visual control scores

Task	PPO	SAC	CURL	DrQ-v2	Dreamer
Environment steps	1M	1M	1M	1M	1M
Acrobot Swingup	3	4	4	166	**229**
Ball In Cup Catch	829	176	**970**	**928**	**972**
Cartpole Balance	516	937	**980**	**992**	**993**
Cartpole Balance Sparse	881	**956**	**999**	**987**	**964**
Cartpole Swingup	290	706	771	**863**	**861**
Cartpole Swingup Sparse	1	149	373	**773**	**759**
Cheetah Run	95	20	502	716	**836**
Finger Spin	118	291	**880**	**862**	589
Finger Turn Easy	253	200	340	525	**878**
Finger Turn Hard	79	94	231	247	**904**
Hopper Hop	0	0	164	**221**	**227**
Hopper Stand	4	5	777	**903**	**903**
Pendulum Swingup	1	592	413	**843**	744
Quadruped Run	88	54	149	450	**617**
Quadruped Walk	112	49	121	726	**811**
Reacher Easy	487	67	689	**944**	**951**
Reacher Hard	94	7	472	670	**862**
Walker Run	30	27	360	539	**684**
Walker Stand	161	143	486	**978**	**976**
Walker Walk	87	40	822	768	**961**
Task mean	94	81	479	770	**861**
Task median	206	226	525	705	**786**

Table 12: DeepMind Control Suite scores under visual inputs.

BSuite performance spectrum

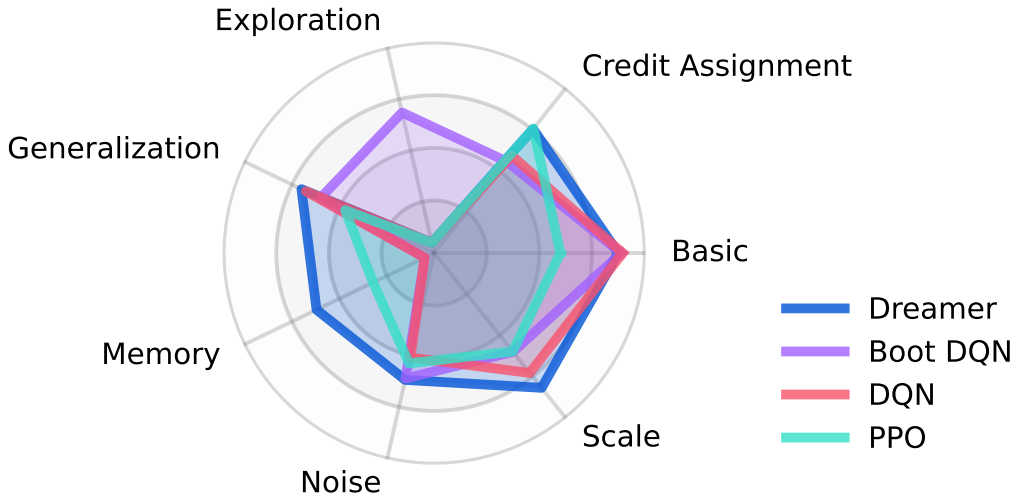


Figure 16: BSuite scores visualized by category[48]. Dreamer exceeds previous methods in the categories scale and memory. The scale category measure robustness to reward scales.

BSuite scores

Task	Random	PPO	AC-RNN	DQN	Boot DQN	Dreamer
Bandit	0.00	0.38	**1.00**	0.93	**0.98**	**0.96**
Bandit Noise	0.00	0.61	0.63	0.71	**0.80**	0.75
Bandit Scale	0.00	0.39	0.60	0.74	**0.83**	0.78
Cartpole	0.04	0.84	0.40	0.85	0.69	**0.93**
Cartpole Noise	0.04	0.77	0.20	0.82	0.69	**0.93**
Cartpole Scale	0.04	0.83	0.12	0.72	0.65	**0.92**
Cartpole Swingup	0.00	0.00	0.00	0.00	**0.15**	0.03
Catch	0.00	0.91	0.87	0.92	**0.99**	**0.96**
Catch Noise	0.00	0.54	0.27	0.58	**0.68**	0.53
Catch Scale	0.00	**0.90**	0.17	0.85	0.65	**0.94**
Deep Sea	0.00	0.00	0.00	0.00	**1.00**	0.00
Deep Sea Stochastic	0.00	0.00	0.00	0.00	**0.90**	0.00
Discounting Chain	0.20	0.24	**0.39**	0.25	0.22	**0.40**
Memory Len	0.00	0.17	**0.70**	0.04	0.04	0.65
Memory Size	0.00	0.47	0.29	0.00	0.00	**0.59**
Mnist	0.05	0.77	0.56	**0.85**	**0.85**	0.61
Mnist Noise	0.05	**0.41**	0.22	0.38	0.34	0.34
Mnist Scale	0.05	**0.76**	0.09	0.49	0.31	0.55
Mountain Car	0.10	0.10	0.10	**0.93**	**0.93**	**0.92**
Mountain Car Noise	0.10	0.10	0.10	**0.89**	0.82	**0.87**
Mountain Car Scale	0.10	0.10	0.10	0.85	0.56	**0.90**
Umbrella Distract	0.00	**1.00**	0.09	0.30	0.26	0.74
Umbrella Length	0.00	**0.87**	0.43	0.39	0.39	0.78
Basic	0.04	0.60	0.58	**0.90**	**0.89**	**0.88**
Credit assignment	0.03	**0.76**	0.37	0.59	0.56	**0.75**
Exploration	0.00	0.00	0.00	0.00	**0.68**	0.01
Generalization	0.06	0.47	0.19	**0.68**	0.60	**0.70**
Memory	0.00	0.32	0.49	0.02	0.02	**0.62**
Noise	0.02	0.54	0.24	0.51	**0.61**	**0.62**
Scale	0.04	0.60	0.22	0.73	0.60	**0.82**
Task mean (%)	3	49	32	54	60	**66**
Category mean (%)	3	47	30	49	57	**63**

Table 13: BSuite scores for each task averaged over environment configurations, as well as aggregated performance by category and over all tasks.

Robustness ablations

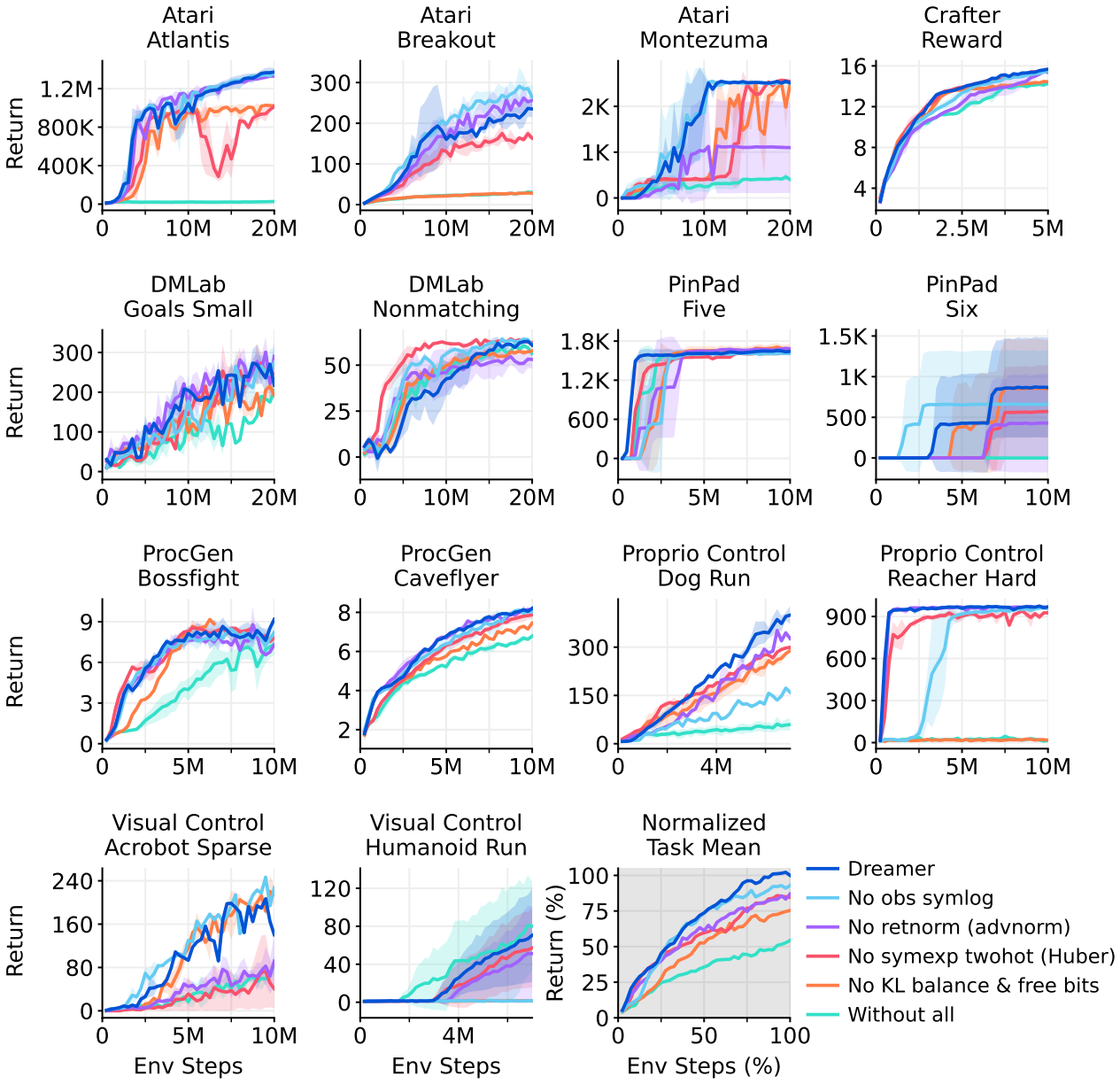


Figure 17: Individual learning curves for the robustness ablation experiment. All robustness techniques contribute to the overall performance of Dreamer, although each individual technique may only improve the performance on a subset of the tasks.

Learning signal ablations

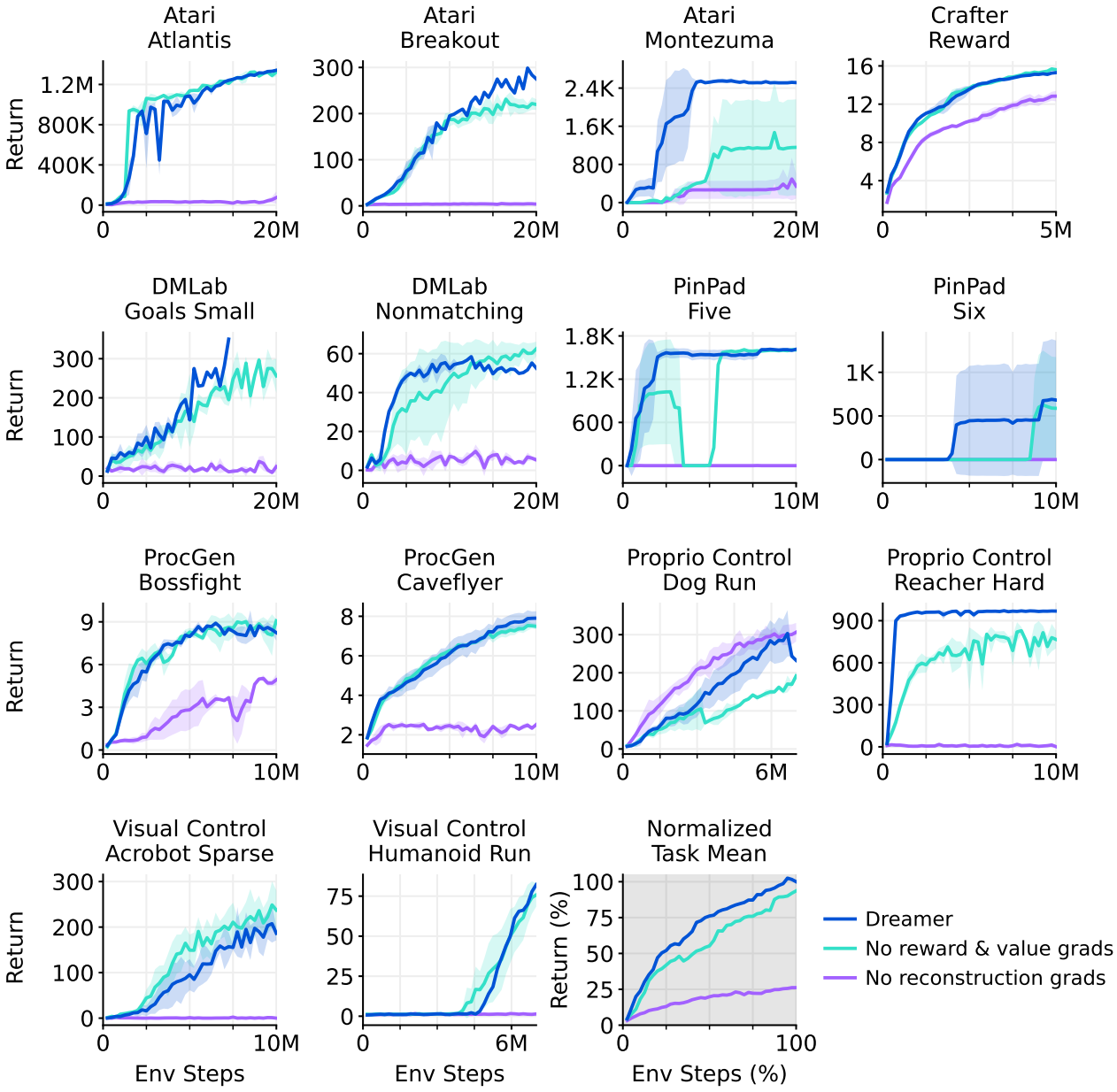


Figure 18: Individual learning curves for the learning signal ablation experiment. Dreamer relies predominantly on the undersupervised reconstruction objective of its world model and additional reward and value gradients further improve performance on a subset of tasks.